

The Cauldron Protocol

Independent Security Audit

MagicFrens — Uniswap v4 hook, eternal-relaunch registry,
hook-native perpetuals, and progressive launch seeder

Deployment target: Robinhood Chain — an Arbitrum Nitro / Orbit L2

Test harness: Uniswap v4 on an Ethereum-Sepolia fork (L1-like)

Scope: contracts/solidity/ excluding lib/, out/, cache/, vendor/

Corpus: 23 in-scope contracts and libraries, ~9,200 lines, 735 external/public ABI
entries

Result summary

Severity	Found	Fixed in this pass
Critical	0	—
High	2	0 (both design-level; remediation specified)
Medium	7	5
Low	6	4
Info	3	0
Total	18	9

Test suite after remediation: 340 / 340 passing

312 pre-existing + 28 new (attack, fuzz and handler-driven invariant tests added by this audit)

Contents

1	Executive summary	2
2	Scope, method and evidence standard	4
2.1	Scope	4
2.2	Method	4
2.3	Evidence standard	4
3	System architecture	6
3.1	The lifecycle	6
3.2	Interaction graph	6
3.3	The two ledgers	6
4	Invariant register	8
5	Findings	12
5.1	F-01 — Protocol randomness relies on <code>blockhash</code>	13
5.2	F-18 — Success voids the exit guarantee at the reserve ceiling	15
5.3	F-02 — Auto-migrate consent is irrevocable	17
5.4	F-03 — Legacy sweep silently forgets owed value	18
5.5	F-04 — Unbounded funding rate inverts or bricks the perp engine	19
5.6	F-05 — Seeder range-cap fallback can mis-side a band	20
5.7	F-09 — The dividend hook is gas-starvable	21
5.8	F-19 — A zero <code>emergencyDelay</code> silently voids the forced-open exit	23
5.9	F-06, F-07, F-08 — reserve and migration hygiene	25
5.10	F-10 — <code>uint32</code> epoch rollover panics the TWAP oracle	26
5.11	F-11, F-12 — L2 timing findings (documented, not fixed)	28
5.12	F-13 to F-17 — lower-severity and informational	30
6	L2 deployment considerations (Arbitrum Nitro / Orbit)	32
6.1	Gas reserves	32
6.2	<code>tx.origin</code> attribution	33
6.3	Reorgs and finality	33
7	Attacks constructed and refuted	34
8	Test suite added by this audit	37
8.1	Reproduction	38
9	Pre-deployment checklist	39
A	Registry / facet storage layout verification	40
B	Complete function inventory	41

1 Executive summary

The Cauldron is an unusually tightly-coupled system. A single Uniswap v4 hook is simultaneously a fee engine, a volume oracle, a death detector, an NFT gacha, a liquidation keeper and a liquidity streamer; a registry orchestrates an endless cycle of token generations whose entire supply lives inside two v4 positions; a perpetuals engine executes real pool swaps from inside that hook's `afterSwap`; and a delegatecall facet runs against the registry's storage to keep the registry under EIP-170. Value therefore moves through call graphs that are several frames deep and partly re-entrant by design.

Given that, the headline result is a good one. **No critical, fund-draining defect was found.** The properties that matter most — 1:1 migration conservation, the reserve's coverage of every claimant at a relaunch, non-inflatability of the ERC-20, the structural separation of the seeder from the redemption reserve, and the byte-identity of the registry/facet storage layouts — were each checked against the code and hold. Several of them hold *structurally* rather than by careful bookkeeping, which is the stronger form. Section 4 argues each one from cited code, and Section 7 records the attacks that were constructed and then refuted.

What the review did find is a cluster of *accounting-integrity* and *liveness* defects, plus one design-level issue rooted in the deployment target:

- **Success voids the exit guarantee** (F-18, **High**). The migration reserve is a single-sided token band parked about $69\times$ below launch. If the token appreciates *through* that ceiling the band goes in-range, stops delivering pure token, and every exit — 1:1 migration, the OG redemption floor, and each collection floor — reverts outright. It cannot be repaired for the live generation: the ceiling is fixed at seed time, and the only way to re-place a reserve is a relaunch, which requires the pool to be *dead*. A token that just went $69\times$ will not be. Reproduced on a live pool; remediation specified.
- **The protocol's randomness is documented-unsafe on the target chain** (F-01, **High**). Three separate mechanisms — the crystal gacha, the NFT rarity reveal, and the anti-snipe surtax jitter — bottom out in `blockhash`. Arbitrum documents that value as “a cryptographically insecure, pseudo-random hash” whose values “do not come from L1”, and states plainly that “Arbitrum's child chain block hashes should not be relied on as a secure source of randomness.” The gacha mints saleable NFTs, so this is an economic, not a cosmetic, dependency. No code fix is shipped here because none fits: the hook has 88 bytes of EIP-170 headroom. The remediation is a configuration and wiring decision, specified in the finding.
- **Two permanent-brick paths in the perpetuals engine** (F-04, F-10). Both terminate in the same place: `_pokeFunding` reverting. Every mutating perp entrypoint calls it — including `forceCloseAllDead`, whose failure leaves `openCount` non-zero forever, which makes `syncGeneration` revert `PositionsOpen` forever, stranding the entire perp liquidity vault in a dead generation. One path was an unbounded governance parameter; the other was an epoch-rollover panic the fuzzer found, caused by copying Uniswap's `uint32` timestamp packing without copying its unchecked arithmetic. Both are fixed.
- **An irrevocable authorisation to burn a wallet's balance** (F-02). `enableAutoMigrate` had no counterpart. The flag is the only thing gating `autoMigrateBatch`, which burns a holder's entire old-generation balance with no allowance, permissionlessly, at any future relaunch. That directly contradicts the registry's own stated model that “migration is a choice, not a requirement.” Fixed.
- **A silently-forgotten value counter** (F-03) and **a gas-starvable fee hook** (F-09). Both were confirmed by writing the attack first and watching it succeed against the unmodified contracts; F-09's exploit window was measured empirically at 117,500–130,000 gas. Both are fixed, and the tests fail against the pre-fix bytecode.

Nine findings were fixed in-place during this engagement, each with a regression test that was verified to *fail* against the pre-fix code. The remaining nine are documented with concrete remediation; the two High findings and four others are deliberately left as deployment-configuration or design decisions rather than code changes, for reasons stated in each finding — in both High cases the honest remedy is larger than an auditor should land unilaterally, and in F-01’s case it does not fit in the hook’s remaining 88 bytes.

A note on what this review could not do

The Foundry harness forks Ethereum Sepolia, where `block.number` and `block.timestamp` advance together. That chain cannot reproduce Arbitrum’s semantics. Every L2 conclusion in Section 6 is therefore reasoned analytically from Arbitrum’s published behaviour and then, where it was possible to do so honestly, *simulated* by advancing `vm.roll` and `vm.warp` independently. Those simulations demonstrate the protocol’s response to a decoupled clock; they do not prove behaviour on the real sequencer. Items that require on-chain confirmation on the target are labelled **Suspected** and listed in the pre-deployment checklist (Section 9).

2 Scope, method and evidence standard

2.1 Scope

Every Solidity file under `contracts/solidity/` except `lib/`, `out/`, `cache/` and `vendor/`. Concretely: `CauldronRegistry.sol`, `CauldronHook.sol`, `CauldronToken.sol`, and all of `cauldron/`. `MagicFrensPeg.sol` and `MagicFrensPresale.sol` are excluded from the build by the cauldron Foundry profile and are not part of this system. In the table below, paths under `cauldron/` are shown without the prefix.

File	LoC	Role
<code>CauldronHook.sol</code>	1854	v4 hook: fees, volume clock, gacha, liquidation keeper, seeder nudge
<code>CauldronRegistry.sol</code>	1392	Lifecycle orchestrator; owns both v4 positions
<code>PerpEngine.sol</code>	1361	Hook-native longs/shorts with real price impact
<code>PoolOps.sol</code>	805	Linked library: all v4 <code>PositionManager</code> encoding
<code>MiFrensGenesis.sol</code>	582	Genesis <code>ERC721Votes</code> collection + ignition
<code>CauldronSeeder.sol</code>	435	Progressive (streamed) launch liquidity
<code>CauldronGachaRouter.sol</code>	369	One-click swap + crystal play
<code>CauldronCollection.sol</code>	357	Per-brew <code>ERC721C</code> volume collection
<code>CauldronGovernor.sol</code>	334	Permissionless proposals, checkpointed votes
<code>PerpVault.sol</code>	326	Community two-sided perp liquidity vault
<code>CauldronBase.sol</code>	316	Shared storage for registry + facet
<code>MigrationVesting.sol</code>	307	Anti-dump migration escrow
<code>MiFrensDividend.sol</code>	279	Genesis fee dividend (“cast the spell”)
<code>CollectionLedger.sol</code>	156	Legacy-floor cap table
<code>RedemptionExt.sol</code>	152	Delegatecall facet of the registry
<code>SeedLib.sol</code>	146	Pure stream schedule + band geometry
<code>CauldronVault.sol</code>	126	Per-brew <code>ETH</code> floor vault (vestigial, see F-14)
<code>ReserveLib.sol</code>	107	Pure reserve tick + liquidity math
<code>LaunchSniper.sol</code>	96	Atomic ignition + funding buy
<code>CauldronFactory.sol</code>	69	Collection/vault deployer
<code>CauldronToken.sol</code>	61	Fixed-supply, non-mintable generation <code>ERC-20</code>
plus <code>RoyaltyRouter.sol</code> , <code>DefaultFeeRouter.sol</code> , <code>PerpStakerOracle.sol</code> and the interface files.		

2.2 Method

The system was read end-to-end before a single finding was written. Pass A built a whole-system model: the cross-contract call graph, the money flows, and an explicit invariant register. Pass B then attacked that model, taking each invariant in turn and asking what would have to be true to break it. Findings were only written once the attack was either constructed concretely or positively refuted from cited code.

2.3 Evidence standard

Every claim in this report is anchored to a file, a function, and the specific lines reasoned about. Three labels are used and they are not interchangeable:

- **Confirmed** — demonstrated by a test that fails against the unmodified contract and passes against the fixed one, or established directly from cited code with no unresolved assumption.
- **Suspected** — the code path is real and the reasoning is sound, but the conclusion depends on behaviour of the deployment chain that this harness cannot execute. Never stated as fact.

- **Refuted** — an attack was constructed and shown not to work. Recorded in Section 7 with the safety argument, because a refuted attack is evidence too.

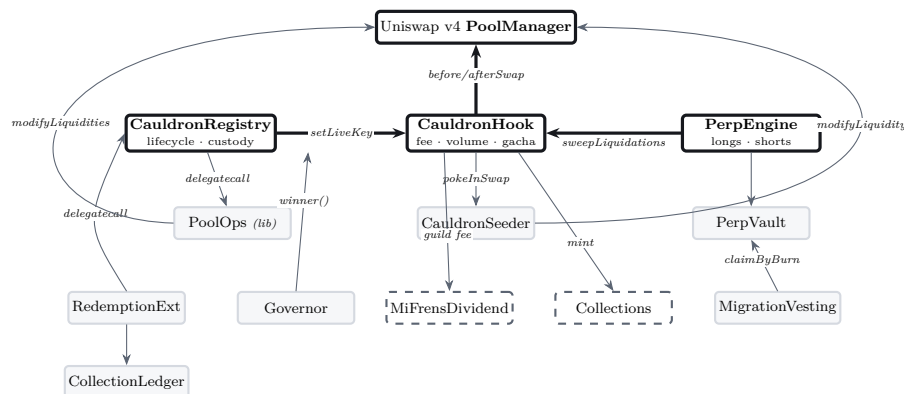
3 System architecture

3.1 The lifecycle

The protocol is a loop, not a product. One iteration:

1. **Ignition.** MiFrensGenesis sells the OG tranche. On sellout, `finalize()` forwards the whole treasury to `CauldronRegistry.summon()`. Ignition is a dedicated role (*igniter*), so registry *ownership* can stay with the governance timelock.
2. **Summon.** The registry deploys a fixed-supply `CauldronToken` via plain `CREATE`, initialises a v4 pool, and seeds *two* positions: an **active** band and an out-of-range single-sided **reserve** holding the migration + genesis supply. Two seed paths exist: the default atomic “green candle” (seed everything, buy the reserve back out — a real first-block market buy) and the opt-in **progressive** stream via `CauldronSeeder`.
3. **Life.** Swaps pay an ETH-denominated fee taken by the hook through v4 return deltas. The fee splits across the genesis dividend, the collection floor (as token buy pressure), the proposer, and the relaunch reserve. Volume accrues crystal credit, which forges gacha NFTs. The perp engine opens leverage against the pool’s real depth.
4. **Death.** The hook’s rolling 24-hour, **wall-clock** volume window falls below `deathThreshold`.
5. **Relaunch.** Permissionless. Force-close every perp, tear down both positions, burn the recovered tokens, pull the hook’s fee reserve, sweep the vault, launch the governor’s winning proposal, and seed the newborn — sizing its reserve to cover 1:1 migration *plus* the unclaimed genesis floor *plus* every collection’s legacy entitlement.

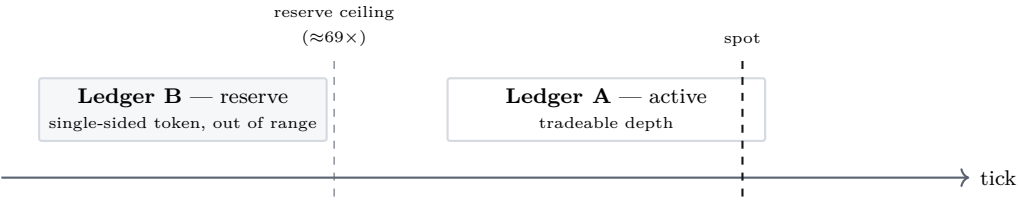
3.2 Interaction graph



Reading the graph. Bold edges are the re-entrant ones and are where the interesting risk lives. A single user swap can, within one `PoolManager` unlock, reach: the hook’s `afterSwap` → a nested legacy-buyback swap → the seeder’s core `modifyLiquidity` → the perp engine’s in-lock liquidation settlement swaps → an NFT mint. Five contracts mutating state and settling v4 deltas inside one lock. Section 7 records why that composition is nevertheless delta-safe.

3.3 The two ledgers

The single most load-bearing structural decision in the protocol is the separation of **ledger A** (the tradeable active band) from **ledger B** (the out-of-range reserve).



Registry owns B. Seeder may only ever hold A. B leaves *only* against a burn (migration) or a floor debit (redemption).

Everything that could drain the protocol reduces to one question: *can anything move value out of ledger B other than a 1:1 burn or a properly-debited floor claim?* Invariants R1–R4 in Section 4 answer it.

4 Invariant register

Each invariant is stated, then argued from cited code, then given a verdict. “Holds structurally” means the code cannot express the violation, which is stronger than “holds because the arithmetic is careful.”

S1 — Supply is fixed and never inflates

Statement. A generation’s ERC-20 supply is minted once and can only ever decrease.

Argument. `CauldronToken.sol:36-47`: the entire supply is minted in the constructor to the registry. There is no `mint` function, no owner, and no role that can add supply. The only supply-changing entry is `burn(address,uint256) (:58)`, gated `onlyRegistry`. The contract inherits plain `OpenZeppelin ERC20` with no hooks that could mint.

Verdict. **Holds structurally.** The violation is not expressible.

S2 — Migration is exactly 1:1 and non-duplicable

Statement. Migrating n old-generation tokens destroys exactly n and releases exactly n of the live token; the same tokens cannot migrate twice.

Argument. `PoolOps.sol:656-666` (`migrateOne`) burns `amount` from the holder and then pulls the same amount from the reserve. The burn is real supply destruction, so the migration right cannot be replayed — it simply follows whoever holds the old token. The critical detail is line 665:

```
| if (got + CLAIM_DUST < amount) revert("reserve short");
```

Without it, `claimFromReserve` caps at the position’s liquidity and returns *short without reverting*, which would let a holder burn in full and receive less. `CLAIM_DUST` is 10^{12} wei, covering only Uniswap’s round-down in liquidity units.

Verdict. **Holds**, with a bounded dust tolerance of 10^{-6} token per claim. See F-15 for the accumulation argument.

R1 — The relaunch reserve covers every claimant exactly

Statement. At each rebirth the new reserve is at least (old circulating supply) + (unclaimed genesis floor) + (total legacy entitlement).

Argument. This is the one worth doing algebraically. Write T for `TOTAL_SUPPLY`, L for `tokensFromLP`, G for `genesisReserveOutstanding`, E for `collectionLedger.totalEntitled()`.

`CauldronRegistry.sol:664-672` recovers L from the dead positions and burns it, so the old generation’s remaining supply — exactly the migration demand — is $T - L$. Then at `:760-790`:

$$A = L - G - E, \quad R = T - A = (T - L) + G + E.$$

The reserve is therefore *exactly* the sum of the three claims, not an approximation. Order matters and is correct: `_flushLegacyAtRelaunch` and `crystallizeCollection` both run *before* E is read at `:774`, so late-arriving entitlements are covered.

Verdict. **Holds exactly** on the main path, as an accounting identity. Note carefully that this is a statement about the reserve’s *size*, not about its *deliverability* — see **F-18**, where the same reserve stops being withdrawable as token despite being correctly sized. **Two degenerate fallbacks break even the sizing** — one at `:763` ($L = 0$) and one at `:789` ($E \geq A$) — both of which hard-set A to 80% of supply and under-collateralise the reserve. See **F-06**.

R2 — The genesis floor only rises, or pays out its backing

Statement. `floorPerFren()` is monotonically non-decreasing across a recycle/resale cycle, and every payout is backed by a real reserve withdrawal.

Argument. `floorPerFren() = genesisReserveOutstanding / genesisShares` (CauldronBase.sol:301-305). A recycle debits F and withdraws F (RedemptionExt.sol:67-80); the matching resale credits $2F$ and deposits $2F$ (:97-101). Net $+F$ per cycle with the share count constant, so the floor strictly ratchets. Both directions revert rather than settle short: the redeem at :80, and the deposit now at :150 after **F-07**.

Verdict. **Holds** (F-07 closed the one path where a payment was collected without the matching deposit landing).

R3 — A legacy credit never out-runs its reserve backing

Statement. `CollectionLedger.totalEntitled` never exceeds tokens actually deposited into the reserve or covered by the reserve's sizing.

Argument. The design is deliberate and good. The hook cannot deposit into the reserve inline — it runs nested inside `afterSwap` with the `PoolManager` locked — so `legacyBuyStep` *holds* the bought tokens and records `legacyOwedToReserve` (CauldronHook.sol:831). The credit is only issued by `materializeLegacyReserve`, which deposits and credits in one step (RedemptionExt.sol:123-136), crediting exactly the swept amount. At relaunch the unmaterialised remainder is credited as a pure number and covered by R1's sizing.

Verdict. **Holds** — but it depended on the swept amount being reported honestly, which **F-03** broke in the other direction (value forgotten rather than over-credited). Now fixed and covered by invariant L-1 in the new test suite.

R4 — The seeder can only ever touch ledger A

Statement. The progressive seeder can never reach the redemption reserve.

Argument. Structural. The reserve is minted by the registry through the periphery `PositionManager` and the position NFT is owned by the registry (PoolOps.sol:479-518). The seeder places *core* `poolManager.modifyLiquidity` positions owned by *itself* at salt 0 (CauldronSeeder.sol:271-281), and its teardown iterates only its own tracked `ranges` (:291-299). There is no code path by which the seeder names the registry's position id.

Verdict. **Holds structurally.**

P1 — PLV principal sits behind collateral and insurance

Statement. Depositor principal is only reduced after a position's own collateral and the insurance buffer are exhausted.

Argument. Both bad-debt paths route through the buffer first: `_replenishPlv` (PerpEngine.sol:1103-1107) for longs and `_absorbPlvLoss` (:1112-1118) for shorts, each taking `min(loss, insuranceEth)` before touching `plv`, and `_absorbPlvLoss` saturating at zero so an extreme gap cannot underflow. `skimInsurance` (:1331-1338) protects the greater of the configured floor and a risk-scaled minimum, so the buffer cannot be emptied while positions depend on it.

Verdict. **Holds** for the loss paths. **Does not hold for funding** — see P2.

P2 — Funding is a zero-sum transfer

Statement. What the crowded side pays equals what the underweight side receives.

Argument. It does not. `PerpEngine.sol:901-914`: the paying side is capped by its own residual (if `(pay > residual) pay = residual`), and for *every* liquidated long residual is zero because `repay = proceeds` leaves nothing. The receiving side draws its full claim, capped only at `maxFundingBps` (50%) of collateral, from insurance and then from `plv`. The asymmetry is a net PLV outflow.

The code acknowledges this in its own comment and mitigates it by draining insurance first. That is a real mitigation, not a fix. The remaining exposure is bounded by the funding rate and

by `maxFundingBps`, which is why the unbounded `fundingRateBpsPerDay` (**F-04**) mattered so much: it was the one knob that could turn a slow, bounded leak into an unbounded one.

Verdict. **Does not hold; bounded by design.** Documented, with the bound now enforced. See F-04 and F-16.

V1 — Registry and facet storage layouts are byte-identical

Statement. `CauldronRegistry` and `RedemptionExt` agree slot-for-slot, as the `delegatecall` pair requires.

Argument. Verified mechanically, not assumed. Both derive from `CauldronBase` and declare no additional state. Comparing the compiled `storageLayout` of both artifacts yields **53 entries, identical in label, slot, offset and type** (reproduced in Appendix A). The three former immutables (`poolManager`, `positionManager`, `hook`) are correctly *storage* at slots 41–43; as immutables they would resolve to zero inside the facet, since immutables live in the executing contract’s own code.

One subtlety worth recording: OpenZeppelin 5.6’s `ReentrancyGuard` uses an ERC-7201 *namespaced* slot, not a sequential one, so it appears in neither layout and is shared correctly under `delegatecall`. That is why the registry’s forwarders must *not* carry `nonReentrant` — they would double-lock against the facet’s own guard. `CauldronRegistry.sol:1081–1084` states this and `:1089–1109` implements it correctly.

Verdict. **Holds, mechanically verified.** See **F-13** for a tooling gap in the documented verification procedure.

V2 — Every v4 delta is settled

Statement. No path leaves a `PoolManager` delta unsettled or double-settled.

Argument. Four contracts settle inside a single unlock. Each was checked:

- `PoolOps.executeBuy (:361–388)` settles `ethIn` and takes `got`, both read from the returned delta rather than from the intent.
- `CauldronHook.legacyBuyStep (:800–833)` settles the *realised* debit `-d.amount0()`, not the intended `amt`, and returns the unspent remainder to the buffer. Settling the intent would leave a positive un-taken delta and revert the user’s parent swap.
- `PerpEngine._swapBody (:968–992)` settles and takes both legs, and correctly runs *in place* when `_inLocked` rather than re-entering `unlock`, which would revert.
- `CauldronSeeder._settle (:343–358)` nets both `modifyLiquidity` deltas and settles the aggregate, handling each currency in either direction, with `sync` before the ERC-20 settle.

The hook’s fee take is the standard v4 pattern: `poolManager.take` of exactly `total`, offset by returning the same `total` as the hook delta on the unspecified currency (`CauldronHook.sol:1103, :750`).

Verdict. **Holds** on all four paths.

L1 — The death clock is denominated in wall-clock time

Statement. Pool retirement depends on elapsed seconds, not block count.

Argument. `CauldronHook.sol:136–138, 1134–1192`: the window uses `SECONDS_PER_HOUR / SECONDS_PER_DAY` against `_lastUpdateTs`, a `block.timestamp`. This is the correct choice for the target and the report verifies it by simulation: `test_L2A_DeathClockIsWallClockNotBlockCount` advances 50,000 blocks with time frozen (nothing dies) and then a day of wall-clock with only parent-chain-sized block motion (it dies on schedule).

Verdict. **Holds; L2-correct.**

L2 — Per-block guards remain per-block on the target

Statement. Guards that must bound activity per block still do so when `block.number` is the parent chain's.

Argument. Split verdict.

- The perp liquidation throttle is keyed on `block.timestamp` (`PerpEngine.sol:206, 637, 726`) — **correct**, and the code says exactly why. On Orbit a `block.number` key would let one cap span dozens of child blocks.
- The anti-snipe surtax is keyed on `block.number` (`CauldronHook.sol:1046`) — **quantised**. It keeps its intended *wall-clock* length but loses per-block resolution. See **F-11**.

Verdict. **Partially holds.**

5 Findings

ID	Sev	Title	Conf.	Status
F-18	High	Success voids the exit guarantee at the reserve ceiling	Confirmed	Documented
F-01	High	Protocol randomness relies on <code>blockhash</code> , documented unsafe on the target chain	Suspected	Documented
F-02	Medium	Auto-migrate consent is irrevocable	Confirmed	Fixed
F-03	Medium	Legacy sweep silently forgets owed value	Confirmed	Fixed
F-04	Medium	Unbounded funding rate inverts or bricks the perp engine	Confirmed	Fixed
F-05	Medium	Seeder range-cap fallback can mis-side a band and halt the stream	Confirmed	Fixed
F-09	Medium	Dividend hook is gas-starvable: fee bypass and value transfer	Confirmed	Fixed
F-11	Medium	Anti-snipe surtax is quantised to parent blocks	Confirmed	Documented
F-12	Medium	MIN_TWAP of 1 s is not a meaningful floor on a fast L2	Confirmed	Documented
F-19	Medium	A zero <code>emergencyDelay</code> silently voids the forced-open exit	Confirmed	Fixed
F-06	Low	Silent under-collateralised reserve fallback	Confirmed	Fixed
F-07	Low	Payment collected when the reserve deposit cannot land	Confirmed	Fixed
F-08	Low	One short holder reverts the whole auto-migrate batch	Confirmed	Fixed
F-10	Low	<code>uint32</code> epoch rollover panics the TWAP oracle	Confirmed	Fixed
F-14	Low	EIP-170 headroom is nearly exhausted	Confirmed	Documented
F-16	Low	<code>enableAutoMigrate</code> keeps overpayment	Confirmed	Documented
F-13	Info	Documented layout-verification command does not run	Confirmed	Documented
F-15	Info	Funding overflow at an extreme mark	Suspected	Documented
F-17	Info	<code>CauldronVault.redeem</code> is permanently unreachable as configured	Confirmed	Documented

5.1 F-01 — Protocol randomness relies on blockhash

F-01	High	Confidence: Suspected (chain-dependent)	Status: documented
-------------	-------------	--	---------------------------

Location. CauldronHook.sol:1732, 1754 (crystal gacha); CauldronCollection.sol:215, 232 and MiFrensGenesis.sol:417, 434 (rarity reveal); CauldronHook.sol:1064 (surtax jitter).
--

What the code does

Three independent mechanisms derive an outcome from `blockhash`. The gacha is the economically significant one:

```
// CauldronHook._resolveTickets, lines 1731-1754
if (block.number <= b.commitBlock) break; // maturity gate
bytes32 bh = blockhash(b.commitBlock);
...
uint256 roll = uint256(keccak256(abi.encodePacked(bh, player, bi, r))) % 10_000;
bool win = (forced || roll < odds) && minted < max;
if (win) { ... ICauldronCollection(col).mint(player); }
```

The design is a proper commit–reveal: `oddsBps` is fixed at commit, `commitBlock` is the current block, and the seed is a value that does not exist yet. On Ethereum that is sound.

Why the target chain changes the conclusion

Arbitrum’s own documentation is explicit on all three points that matter:

- `blockhash(x)` “[r]eturns a cryptographically insecure, pseudo-random hash for `x` within the range `block.number - 256 <= x < block.number`”;
- “The hashes returned do not come from L1”;
- “Arbitrum’s child chain block hashes should not be relied on as a secure source of randomness”, with Chainlink VRF given as the recommended alternative.

The Robinhood Chain documentation repeats this: “`blockhash(n)` is supported but is only reliable for recent blocks”, and warns against using it as a randomness source.

Compounding it, `block.number` on the target “returns an estimate of the L1 (Ethereum) block number ...and updates only periodically.” So the maturity gate `block.number > commitBlock` is satisfied roughly once per parent block, and every child block inside that window shares one `commitBlock` value — which means one seed governs every crystal committed across dozens of child blocks.

Impact

If the pseudo-random derivation is predictable — which is what “cryptographically insecure” warns of — then the roll is fully determined at commit time, because it is a pure function of four values a player either controls or can compute. A player can then commit only when the outcome is a win. The same argument applies to `reveal()`, where the payoff is a rarity tier with direct secondary-market value.

Even without predictability, the single sequencer’s first-come-first-served ordering gives whoever operates it discretion over which transactions land in which block, and therefore over which seed each commit is anchored to.

Evidence

`test_L2C_RollIsFullyDeterminedByBlockhash` in `test/final/F02_L2Semantics.t.sol` makes the dependency machine-checked: it commits a batch, matures it, reads `blockhash(commitBlock)`, and

shows that recomputing $\text{keccak}(bh, \text{player}, bi, r) \bmod 10^4$ off-chain reproduces the contract's roll exactly. It asserts a property, not an exploit: *the outcome carries no entropy beyond the blockhash*.

Why no code fix is shipped here

CauldronHook has **88 bytes** of EIP-170 headroom (24,488 / 24,576). Every structural remedy — an entropy accumulator fed by each swap, a pluggable `IEntropy` module, a two-phase reveal — costs several hundred bytes. Shipping a half-measure that fits would be worse than documenting the real one.

Remediation, in priority order

1. **Verify the derivation on the target before launch.** Deploy a one-function probe that records `blockhash(block.number - 1)` over a few hundred blocks and check whether the sequence is a function of the block number alone. This is a half-hour task and it decides everything below. Until it is done, treat the gacha odds as advisory.
2. **If predictable: make the seed a third party's future action.** Reclaim bytes first (F-14 lists candidates — CauldronVault integration is vestigial per F-17), then add a hook-level entropy accumulator updated on every swap and require a batch's resolve to consume an accumulator value written *after* its commit. The seed then depends on other traders' behaviour, which the committer cannot predict or produce alone.
3. **Alternative: wire Chainlink VRF** if available on the target, via a policy module rather than in the hook, so the hook's size is unaffected.
4. **Interim mitigation, zero bytes:** widen the maturity gap. Requiring several parent blocks between commit and resolve does not fix predictability, but it removes same-block commit-and-resolve and raises the cost of any ordering game.

Not affected: `prevrandao`

Worth stating explicitly, because it is the more common version of this bug. Arbitrum returns **the constant 1** for both `block.prevrandao` and `block.difficulty`. A grep of the entire in-scope corpus finds **zero** uses of either. `CauldronHook.sol:1055-1061` documents that this was a deliberate migration away from `prevrandao`. That is correct, and it is the right instinct applied one step short of the conclusion — the replacement (`blockhash` plus the live tick) inherits the same class of weakness.

5.2 F-18 — Success voids the exit guarantee at the reserve ceiling

F-18 High Confidence: Confirmed (reproduced on a live pool) Status: documented

Location. ReserveLib.sol:50-66 (reserveTicks), Pool10ps.sol:555-584 (claimFromReserve), :665 (migrateOne's short guard), RedemptionExt.sol:80, CauldronRegistry.sol:180-182 (setReserveCeiling), :1428-1434 (floorClaimableNow).

The mechanism

The migration reserve is a single-sided *token* position parked out of range. Its band is placed RESERVE_CEILING_OFFSET ticks below the launch tick — 42,400 by default, about 69× — and ReserveLib is explicit about the orientation:

“the reserve must sit BELOW the launch tick: it is pure token while `currentTick > reserveTickUpper`, and only begins converting to ETH if the token pumps ~69× down into the range.”

That parenthetical is the finding. Everything downstream assumes the band delivers *pure token1*: claimFromReserve sizes a withdrawal with liquidityForTokenOut, which is only the inverse of the position's contents while the range is entirely below spot. Once the token appreciates through the ceiling, the band is *in range*: removing that liquidity yields a mix of ETH and token, so the token delivered falls short of the amount asked for — and every consumer that demands the full amount reverts:

- migrateOne reverts "reserve short" (Pool10ps.sol:665), closing 1:1 migration for *every* holder of *every* prior generation;
- redeemOgFren reverts NoBalance (RedemptionExt.sol:80), closing the OG floor;
- recycleCollectionNFT reverts on the same guard (Pool10ps.sol:780), closing every collection floor.

Those reverts are individually *correct* — each exists so a holder never burns in full and receives less. The problem is what they compose into.

Why this is High rather than a design note

Three things make it serious.

It fires on success, not on failure. Every other risk in this protocol is triggered by something going wrong. This one triggers when the token does exactly what everyone wants. The exit guarantee is advertised without qualification — the registry describes the reserve as what “backs migration/genesis 1:1” — and it silently stops holding at precisely the moment holders most want to use it.

It cannot be repaired for the live generation. setReserveCeiling is explicit that it applies to “the NEXT summon/relaunch” (CauldronRegistry.sol:176-182); the live band's ticks were fixed at seed time and are stored per-generation in reserveTickUpper[gen]. The only way to re-place a reserve is a relaunch — which requires hook.isDead(), i.e. 24-hour volume below the threshold. A token that just appreciated 69× is the least likely thing on the chain to be quiet. The system is therefore self-locking: the condition that breaks the exit is also the condition that prevents the repair.

The break is total, not proportional. It is not that holders receive less; it is that the transaction reverts entirely. There is no partial exit, no degraded-but-working path.

Reproduction

`test_F18_MigrationBreaksAboveTheCeiling` (`test/final/F04_ReserveCeiling.t.sol`) drives it on live Uniswap v4: summon with a genesis bonus, acquire circulating supply while migration works, then buy repeatedly until `slot0.tick` falls to or below `reserveTickUpper(1)`. At that point it asserts, in order:

1. `registry.floorClaimableNow()` returns `claimable == false`;
2. `claimByBurn(1, circulating)` reverts;
3. `redeemOgFren(1)` reverts;
4. the holder's balance and their fren are both untouched.

The companion `test_F18_MigrationWorksBelowTheCeiling` shows both paths succeeding at the same configuration before the pump, so the difference is the price and nothing else. `test_F18_CeilingIsSetBeforeTheSummonOnly` confirms that calling `setReserveCeiling(138000)` afterwards leaves `reserveTickUpper(1)` unchanged.

What the protocol already does about it

Worth crediting: `floorClaimableNow` (`CauldronRegistry.sol:1428`) exists precisely to surface this state — it returns `claimable = tick > reserveTickUpper[g]`. So the condition is known and is exposed to the frontend. What is missing is any *on-chain* remedy: the view tells a holder their exit is closed, but nothing lets them or governance reopen it.

Remediation

No code fix is applied here because every option is a design decision with real economic consequences, and this is the kind of change that should be chosen by the protocol's authors rather than by its auditor. In rough order of cost:

1. **Set the ceiling deliberately, per iteration.** The setter's bounds (4,000 to 138,000 ticks, roughly $1.5\times$ to $10^6\times$) are wide enough that a launch expecting more than $69\times$ can simply say so *before* the summon. This is free, it is already possible today, and it should be an explicit line item in the launch runbook rather than a default nobody revisits. It narrows the window; it does not close it.
2. **Add a governance-gated reserve *re-placement*** — withdraw the reserve position and re-mint it at a band below the new spot, without touching the active book. This is the real fix. It is a custody action, so it belongs behind the existing `armEmergency` / `timelocked` / guardian-veto machinery, and under the exit-guarantee rule already in `_redeemBlocked` the redemption path is forced open while it is armed. Registry headroom is 339 bytes, so this likely needs a `Poo10ps` entry plus a thin forwarder.
3. **Alternatively, make the short path graceful.** Let `claimFromReserve` deliver token *and* the ETH the band converted into, valuing the pair at the mark, so a holder exits at fair value rather than not at all. This changes the meaning of “1:1” and would need to be stated plainly to holders — which is why it ranks below (2).

Until one of these lands, the honest statement of the guarantee is: *migration and the redemption floor are available while the token trades below its per-iteration reserve ceiling*. That qualification should appear anywhere the guarantee is described.

5.3 F-02 — Auto-migrate consent is irrevocable

F-02 Medium Confidence: Confirmed Status: Fixed

Location. CauldronRegistry.sol:1027-1037 (enableAutoMigrate), :1046-1062 (autoMigrateBatch), CauldronBase.sol:259.

The defect

enableAutoMigrate sets `autoMigrate[msg.sender] = true`. No function anywhere in the corpus sets it back to `false` — verified by `grep` across `CauldronRegistry.sol`, `CauldronBase.sol` and `PoolOps.sol`: the only writes are that one line and the reads in `PoolOps.sol:677`.

That flag is the sole gate on `autoMigrateBatch`, which calls `migrateOne` → `ICauldronBurn(prevToken).burn(h, bal)`. Note what that primitive is: `CauldronToken.burn` is `onlyRegistry` and takes an arbitrary `from` (`CauldronToken.sol:58-60`). It needs no allowance. So an opted-in wallet has handed any permissionless keeper standing authority to destroy its entire old-generation balance, at any future relaunch, at a moment of the keeper's choosing.

Why this matters more than it first appears

The registry's own documentation of `claimByBurn` states the model plainly (:931-935): “*OPTIONALLY migrate ... Migration is a choice, not a requirement — old tokens are never frozen, so a holder who prefers a past iteration simply keeps (or trades) it.*”

A holder who paid 0.069 ETH for convenience during iteration 3 and then decided they prefer iteration 7 had no way to stop the machine converting them. Their only escape was to move their balance to a fresh address — which is not a remedy, it is a workaround for a missing function. The consent is also *unpriced in time*: it was granted under one set of conditions and is exercised under another, possibly years later.

Fix applied

```
/// @notice Revoke the hands-off migration consent (audit F-02).
function disableAutoMigrate() external {
    autoMigrate[msg.sender] = false;
    emit AutoMigrateSet(msg.sender);
}
```

Free — revoking a permission must never sit behind a fee — and effective immediately, because `PoolOps.autoMigrateBatch` reads the flag per holder per call. Re-opting in costs the fee again, so this is not a fee bypass.

Tests

`test_F02_AutoMigrateConsentIsRevocable` and `test_F02_RevocationIsSelfServiceOnly` (`test/final/F01_CustodyAndConsent.t.sol`), plus the handler-driven invariant `invariant_C1_ConsentMatchesTheActorsLastChoice` (`test/final/F03_FinalInvariants.t.sol`), which fuzzes opt-in/opt-out sequences across three actors and asserts the on-chain flag always equals that actor's own last instruction — 128,000 calls, no violation.

5.4 F-03 — Legacy sweep silently forgets owed value

F-03 Medium Confidence: Confirmed Status: Fixed

Location. CauldronHook.sol:848-855 (sweepLegacyReserve).

The defect

```
amt = legacyOwedToReserve;
legacyOwedToReserve = 0; // <-- unconditional
uint256 bal = IERC20(token).balanceOf(address(this));
if (amt > bal) amt = bal; // <-- clamp AFTER
if (amt > 0) IERC20(token).transfer(to, amt);
```

The counter is zeroed *before* the transfer is clamped to the live balance. In any state where `balance < owed`, the difference is destroyed: the tokens remain in the hook, are credited to no collection's floor, and no later sweep can find them, because the only record that they were owed has been erased. The function is the sole consumer of that counter (PoolOps.sol:725), so the loss is unrecoverable without a redeploy.

Severity reasoning

Rated Medium rather than High because reaching `balance < owed` requires a desynchronisation between the counter and the hook's token balance, and the two are written together in `legacyBuyStep` (:823-831). It is rated Medium rather than Low because the failure is *silent and permanent*, the function is reachable permissionlessly through `materializeLegacyReserve`, and the value it guards is every NFT holder's floor. A counter whose only job is to remember value should not be able to forget it under any input.

Fix applied

```
uint256 owed = legacyOwedToReserve;
uint256 bal = IERC20(token).balanceOf(address(this));
amt = owed > bal ? bal : owed;
legacyOwedToReserve = owed - amt; // debit exactly what moved
if (amt > 0) IERC20(token).transfer(to, amt);
```

Invariant R3 is preserved in both directions: the registry credits exactly the returned `amt`, so a credit still cannot out-run the reserve, and now the remainder stays claimable by the next sweep.

Evidence that the test catches it

Reverting the fix and re-running:

Test	Result against pre-fix bytecode
test_F03_PartialSweep...	FAIL: remainder still owed: 0 != 60000000000000000000000000000000
testFuzz_F03_SweepDebits...	FAIL: counter debited by exactly the move: 0 != 19998686626960
invariant_L1_NoOwedValue...	FAIL: 1441207360336501672073743 != 2284619084570732192188892

The invariant is the strongest of the three: across 128,000 fuzzed handler calls it showed roughly 8.4×10^{23} base units silently vanishing. All three pass against the fixed contract.

5.5 F-04 — Unbounded funding rate inverts or bricks the perp engine

F-04 Medium Confidence: Confirmed Status: Fixed

Location. `PerpEngine.sol:1258-1267 (setRisk), :504-521 (_pokeFunding).`

The defect

`setRisk` validates every parameter it takes except one:

```
if (_warmup < MIN_TWAP) revert BadParam();
if (!(_ceiling >= 1 && _ceiling <= 10 && _maintBps <= 5000
    && _maxNotBps <= BPS && _maxOiBps <= BPS)) revert BadParam();
...
fundingRateBpsPerDay = _fundingBpsDay; // no bound at all
```

That value is then used in signed 256-bit arithmetic:

```
int256 step = (imbalance * int256(fundingRateBpsPerDay) * int256(dt) * 1e18)
    / (int256(total) * int256(BPS) * int256(1 days));
```

Two distinct failures follow.

Sign inversion. For any value at or above 2^{255} , the `int256` cast wraps negative. `step` flips sign, so the funding index moves the wrong way and *the crowded side gets paid* by the underweight side — through the PLV, via the non-conservative path documented under invariant P2.

Permanent brick. For a merely large value, the checked multiplication overflows and `_pokeFunding` reverts. This is the serious one, because of what calls it: `openLong`, `openShort`, `close`, `liquidate`, `forceCloseDead`, `forceCloseAllDead`, `_doSweep` and `poke` — every mutating entrypoint. A reverting `forceCloseAllDead` means `openCount` never returns to zero, which means `syncGeneration` reverts `PositionsOpen` forever (:783), which strands the entire token-side inventory in a dead generation with no recovery path.

Reachability

Governance only (`onlyOwner`, the timelock on mainnet), so this is a fat-finger and compromised-governance concern rather than an open attack. It is still worth Medium: it is a one-call, irreversible loss of the whole perp subsystem, and every sibling parameter on the same setter is already bounded — the omission looks like an oversight rather than a decision.

Fix applied

```
if (!(_ceiling >= 1 && _ceiling <= 10 && _maintBps <= 5000 && _maxNotBps <= BPS
    && _maxOiBps <= BPS && _fundingBpsDay <= BPS)) revert BadParam();
```

100% of notional per day is already far beyond any defensible funding rate, and it caps P2's structural leak at a knowable rate.

Tests

`test_L2D_FundingRateIsBounded` rejects 2^{255} , `type(uint256).max` and $10^4 + 1$, and confirms the ceiling and the shipped default remain settable. `testFuzz_L2D_PokeSurvivesAnyAcceptedRate` then fuzzes every accepted rate against a year of decoupled clock advance and asserts `poke()` never reverts.

5.6 F-05 — Seeder range-cap fallback can mis-side a band

F-05 Medium Confidence: Confirmed Status: Fixed

Location. CauldronSeeder.sol:370-381 (`_reserveRange`), :241-283 (`_placeStep`).

The defect

The seeder bounds teardown gas by tracking at most `MAX_RANGES = 64` distinct tick bands. At the cap it degrades rather than halting — a deliberate and correct choice — but it degraded to the wrong thing:

```
if (n >= MAX_RANGES) {
  Range storage r = ranges[n - 1]; // the most recently PUSHED range
  return (r.lo, r.hi); // ...whichever side of spot it is on
}
```

`_placeStep` reserves the *ask* band and then the *bid* band, so at the cap both resolve to that same single range. The two sides are then sized with opposite formulas (:262-267): `getLiquidityForAmount1` for the ask (pure `token1`, band *below* spot) and `getLiquidityForAmount0` for the bid (pure `token0`/ETH, band *above* spot).

Feeding an ask amount into a band above spot mints a position the pool settles entirely in ETH. `_settle` is then asked for an ETH debit bearing no relation to `ethStep` — and which the seeder does not hold — so the placement reverts. In-swap that is swallowed by the hook’s gas-bounded call; via the permissionless `poke()` it is a hard revert. Either way the stream halts, which is precisely what the degrade-don’t-halt design existed to prevent.

Reachability

Real, not theoretical. A new range is tracked each time the aligned tick moves one spacing (:254-260 via `SeedLib.askBand/bidBand`). At `spacing = 200`, exhausting 64 ranges takes roughly 64 spacings of tick drift — comfortably inside one launch window of genuine trading, which is exactly the period the progressive seed is designed for.

Fix applied

Track the last band *per side* and fall back to the matching one, so an ask always degrades into an ask and a bid into a bid:

```
function _reserveRange(int24 lo, int24 hi, bool isAsk) private returns (int24, int24) {
  ...
  if (n >= MAX_RANGES) {
    Range storage r = isAsk ? _lastAsk : _lastBid;
    if (r.lo == 0 && r.hi == 0) return (0, 0); // decline rather than mis-side
    return (r.lo, r.hi);
  }
  ...
}
```

Three supporting changes: `_lastAsk`/`_lastBid` are cleared in `startSeed` alongside `ranges` (they are tick coordinates in the previous generation’s pool); the two-sided full-range *base* is pushed directly rather than through `_reserveRange`, so it never becomes either side’s fallback; and `_placeStep` guards `hi > lo` before sizing, since a declined band would otherwise divide by zero.

Note on residual behaviour

At the cap the stream continues into an increasingly stale band. That is the intended degradation and nothing is stranded — the band is tracked, so `withdrawAll` still recovers it. The invariant that matters is preserved: *the seeder never places into an untracked range*.

5.7 F-09 — The dividend hook is gas-starvable

F-09 Medium Confidence: Confirmed (window measured) Status: Fixed

Location. MiFrensGenesis.sol:559-561 (`_update`), MiFrensDividend.sol:224-233 (`onMiFrenTransfer`), :191-194 (the stale branch of `_castSpell`).

The defect

```
if (from != address(0) && dividend != address(0)) {
  try IMiFrensDividendHook(dividend).onMiFrenTransfer(tokenId, from) {} catch {}
}
```

`try/catch` swallows an out-of-gas child exactly as readily as a genuine `revert`, and EIP-150 forwards only 63/64 of the remaining gas. A seller who sizes their transaction’s gas precisely can therefore make this child call run out of gas while the transfer itself still settles.

Why the first attempt at this attack failed, and what fixed it

This is worth recording because it changed the verdict twice.

The obvious version does *not* work. For a **genesis** fren the code after the dividend call includes an `everMoved SSTORE` (:565-567), costing $\geq 20,000$ gas. If the child is starved, the parent resumes with only 1/64 of the gas — a few hundred units — and cannot pay for that write, so the whole transfer reverts. A gas sweep from 60k to 260k in 5k steps confirmed it: at every budget where the transfer succeeded, the hook had run.

The attack works on the **forged** tranche. That `SSTORE` is guarded by `tokenId <= GENESIS_SUPPLY`, so for a volume-minted fren (`id > GENESIS_SUPPLY`) the post-call work is nearly free and the 1/64 remainder covers it. Re-running the sweep against a forged fren:

Gas budget	Transfer settles?	activeShares after
$\leq 115,000$	no	—
117,500 – 130,000	yes	1 ← hook starved
$\geq 132,500$	yes	0 (hook ran)

A deterministic window roughly 13,000 gas wide, hit at will.

Impact

Three consequences, and they are precisely the ones the M-06 comment in that same function says it exists to prevent — reachable on demand:

1. **Permanent dilution.** `enchantedBy[tokenId]` still points at the seller, so the fren stays counted in `activeShares` while earning for nobody. Every honest holder’s share of every future fee is reduced, indefinitely.
2. **Backwards value transfer.** When the buyer eventually re-casts, the stale branch of `_castSpell` credits `owed[cur] += (accPerShare - debtOf[tokenId]) / ACC` — paying the **seller** for the accrual earned while the **buyer** owned the NFT.
3. **Fee bypass.** That stale branch never calls `_collectEnchantFee` (:191-194 versus :183-190), so the buyer re-enchants free, dodging the reserve-growing fee a moved fren is supposed to pay — and forged frens are exactly the tranche that always pays it.

Fix applied

```
if (from != address(0) && dividend != address(0)) {  
    if (gasleft() < GAS_DIVIDEND_MIN) revert InsufficientGas(); // 80_000  
    try IMiFrensDividendHook(dividend).onMiFrenTransfer{gas: GAS_DIVIDEND_FWD}(tokenId, from) {}  
    catch {}  
}
```

The hook's work is a fixed, small set of storage writes — worst case about 39k, dominated by one cold zero-to-non-zero owed SSTORE plus three cold non-zero updates. Requiring 80k before attempting it, and forwarding an explicit 60k, removes the attacker's ability to *choose* failure while keeping the property that a genuinely broken dividend can never brick a transfer: a real revert is still caught. Mints (`from == 0`) never reach this branch, so the in-protocol badge and art mint paths are untouched.

Tests

`test_F09_GasStarvedTransferIsRefused` targets the measured window's midpoint. `test_F09_The-EntireGriefWindowIsClosed` sweeps 110k–132k in 2k steps and asserts the property directly — *a settled transfer always breaks the spell* — so the fix cannot be evaded by retuning gas. Two control tests confirm normal transfers still settle and a reverting dividend still cannot brick one. Against the pre-fix bytecode the first two fail (FAIL: `gas-starved transfer must not settle` and FAIL: `a settled transfer always breaks the spell: 1 != 0`) while both controls pass.

5.8 F-19 — A zero emergencyDelay silently voids the forced-open exit

F-19 **Medium** **Confidence: Confirmed (observed on the live deployment)** **Status: Fixed**

Location. `CauldronBase.sol:311-313` `(_redeemBlocked)`, `CauldronRegistry.sol:250-255`
 `(_consumeTimelock), :159` (the immutable assignment).

The interaction

Two guards that are individually correct compose into a silent hole:

```
// CauldronBase -- THE EXIT GUARANTEE
function _redeemBlocked() internal view returns (bool) {
    return redemptionPaused && emergencyReadyAt == 0;
}
// CauldronRegistry -- the delay is OPTIONAL
function _consumeTimelock() private {
    if (emergencyDelay != 0) { ... emergencyReadyAt = 0; }
}
```

When `emergencyDelay == 0`, `_consumeTimelock` is a no-op, so `armEmergency()` is never required to execute a custody action — and therefore `emergencyReadyAt` is never set. `_redeemBlocked()` collapses to plain `redemptionPaused`, and the protocol’s most prominently-advertised holder protection — *the redemption exit is forced open the instant a custody action is armed, so holders can always leave at floor first* — never engages at all.

The failure is not that the guarantee is weakened. It is that it is **absent while appearing present**: the code reads correctly, the comment describes a real protection, and every test that arms an emergency exercises the working path.

Consequence

The emergency admin can call `setRedemptionPaused(true)` — which is deliberately *not* time-locked, so it is immediate — and then `emergencyWithdrawLP`, with no announcement window and with holders still locked out of redemption. That is precisely the sequence the forced-open exit was written to make impossible.

This is live, not hypothetical

Read from the deployed round-31 registry (0x6629...11b8) on Sepolia:

Getter	Value	
<code>emergencyAdmin()</code>	0x8729...3350	the timelock ✓
<code>emergencyDelay()</code>	0	arm-and-wait inert; exit guarantee void
<code>guardian()</code>	0xC944...c133	set ✓
<code>owner()</code>	0x6486...a1A7	the presale, not the timelock

`emergencyDelay` is immutable, assigned once in the constructor (`CauldronRegistry.sol:159`). It **cannot be corrected on this deployment**. The OpenZeppelin timelock’s own `minDelay` still gates the admin, so actions are not literally instant — but the registry’s second layer, and with it the exit guarantee, is simply not there. The `owner()` reading as the presale independently confirms this deployment predates the ignition-role split.

Fix applied

The root cause is that one configuration value was silently deciding two unrelated things. `emergencyDelay` should govern *how long* the exit window is; it was also, accidentally, governing *whether the window exists at all*. The fix separates them by making the arm unconditional:


```
function _consumeTimelock() private {  
    if (emergencyReadyAt == 0) revert Timelocked(); // must be ARMED, always  
    if (emergencyDelay != 0 && block.timestamp < emergencyReadyAt) revert Timelocked();  
    emergencyReadyAt = 0;  
}
```

Every custody action now requires a prior `armEmergency()` regardless of the configured delay, so `emergencyReadyAt` is always set while one is pending — and the forced-open exit therefore always engages. A zero delay now degrades the window to a transaction boundary instead of deleting the mechanism, which is a safe failure mode rather than a silent one.

Rejecting a zero delay at construction was the alternative. It was not chosen because `emergencyDelay` is immutable: a constructor revert protects future deployments but leaves the *structural* defect in place, so any deployment that slipped through would still have no exit guarantee. Fixing the mechanism is strictly stronger, and it costs one byte of bytecode (the registry went from 24,237 to 24,236 bytes). Choosing a non-zero delay remains a deployment requirement for a real *window*, and stays in the checklist.

Tests

`test_Timelock_ZeroDelay_StillRequiresTheArm` asserts an unarmed custody action is refused even at zero delay, and that an armed one clears the guard immediately. `test_F19_ZeroDelay_ArmingStillForcesTheExitOpen` asserts the property the finding is actually about: with redemption paused and the delay at zero, arming still sets the flag the exit guarantee is keyed on. The first test replaces one that asserted the *opposite* — that a zero delay makes custody actions outright instant — which had encoded the defect as intended behaviour and is why the hole survived earlier review passes.

Eleven existing tests were updated to arm before a custody call. Every one of them was previously exercising the unarmed path without noticing, which is itself evidence of how invisible the defect was.

5.9 F-06, F-07, F-08 — reserve and migration hygiene

F-06	Low	Silent under-collateralised reserve fallback	Status: Fixed
-------------	------------	---	----------------------

CauldronRegistry.sol:763

Invariant R1 holds exactly on the main path, but two degenerate branches abandon it. The legacy branch at :783-788 already emits `ReserveShortfall`. Its twin one line earlier did not:

```
if (newActive == 0) newActive = GEN1_ACTIVE_TOKENS; // ultra-rare fallback
```

This fires when the dead pool returned nothing (`tokensFromLP == 0` — a fully drained book, or a progressive generation whose teardown recovered no token) and hard-sets the active band to 80% of supply. The reserve is then 20% of supply against roughly 100% migration demand. The exit guarantee silently degrades to first-come-first-served and late migrants hit `reserve short`.

It is the more dangerous of the two branches, and it was the silent one. **Fixed** by emitting the same signal, so both shortfall shapes are observable to an indexer. Note this makes the condition *visible*, not impossible — resolving an under-collateralised rebirth is a governance policy question (proportional haircut versus first-come), which is outside a code fix.

F-07	Low	Payment collected when the deposit cannot land	Status: Fixed
-------------	------------	---	----------------------

RedemptionExt.sol:141-151 (<code>_pullGrow</code>)
--

`PoolOps.addToReserve` returns 0 *without reverting* when the amount maps to zero liquidity units (`ReserveLib.liquidityForTokenOut` rounds down, `PoolOps.sol:604-605`) or when the generation has no reserve position at all — reachable via the dust guard at `PoolOps.sol:501`, which leaves `generationReservePositionId[g] == 0`.

By then `transferFrom` has already pulled the caller's tokens. The old code kept the payment, grew the floor by nothing, and — in `buyTreasury0gFren` — still handed over the fren. **Fixed** by reverting when `added == 0`: a donation that cannot reach the reserve must not be collected.

F-08	Low	One holder reverts the whole auto-migrate batch	Status: Fixed
-------------	------------	--	----------------------

PoolOps.sol:671-682

The `NatSpec` promises the batch “skips anyone . . . whom the pool can't currently cover, so one miss never reverts the batch.” Nothing implemented that: `migrateOne` reverts “`reserve short`” and the revert propagates out of the loop, killing the entire keeper call.

Because opting in is permissionless — and, before F-02, irrevocable — a single opted-in wallet holding more than the reserve can pay was enough to make every batch containing it revert. A cheap, durable grief against the hands-off path the protocol advertises. **Fixed** by re-reading the reserve's capacity per iteration and skipping any holder it cannot cover in full, so nobody is partially migrated behind their back either.

5.10 F-10 — uint32 epoch rollover panics the TWAP oracle

F-10	Low	Confidence: Confirmed (found by fuzzing)	Status: Fixed
-------------	------------	---	----------------------

Location. <code>PerpEngine.sol:395-410</code> (<code>_writeObs</code>), <code>:425-464</code> (<code>twapTick</code>).

How it was found

`testFuzz_L2D_PokeSurvivesAnyAcceptedRate` was written to check the F-04 bound. With `dt` unbounded it failed on `args=[2946, 4294967294]` — `panic: arithmetic underflow or overflow`, at 567 gas into `poke()`. Not the bug being looked for, and a real one.

The defect

The oracle packs timestamps as `uint32` — Uniswap’s trick, and a good one, since it lets `Observation` fit (`uint32`, `int56`) in one slot. But Uniswap performs every timestamp *delta* inside an unchecked block, and this code did not:

```
uint32 nowTs = uint32(block.timestamp);
uint32 dt = nowTs - lastObsTs; // checked: panics on wrap
```

Past 2^{32} seconds (07 Feb 2106) `uint32(block.timestamp)` is *smaller* than the stored `lastObsTs`, so the checked subtraction panics instead of yielding the correct modulo- 2^{32} delta. Four further sites in `twapTick` have the same shape (`nowTs - twapWindow`, `nowTs - oldest.ts`, `nowTs - useTs`, `nowTs - lastObsTs`).

Impact

Not a degraded oracle — a permanent brick, by the same chain as F-04: `_writeObs` → `_pokeFunding` → every mutating entrypoint including `forceCloseAllDead` → `openCount` never reaches zero → `syncGeneration` reverts forever. No position can ever be closed. `twapTick` being a `view` does not help: it is reached from `markSqrtPriceX96` → `_quoteMark` → `_underwater`, i.e. from inside every liquidation and settlement.

Severity Low because the date is 80 years out and, importantly, LP principal remains recoverable: `withdrawPlvTo` and `withdrawPlvTokenTo` (:1230-1252) do not call `_pokeFunding`, so `PerpVault` withdrawals still work. Trader positions, however, would be permanently stuck. It is reported at all because the protocol brands itself an eternal machine, the engine has no upgrade path, and the fix costs nothing.

Fix applied

Every timestamp delta is now *unchecked*, restoring Uniswap’s semantics, under which the arithmetic is exact for any span shorter than 2^{32} seconds (136 years). Bytecode *shrinks* slightly, which matters given F-14.

Residual

The binary search in `twapTick` compares `ts <= target` directly rather than with a wrap-aware comparator like Uniswap’s `lte`. Across the epoch boundary that ordering is briefly wrong, for at most one ring lifetime (~ 16 minutes) once every 136 years, and the outcome is a poor-but-bounded mark rather than a panic — `MIN_TWAP` and the `ok` flag still gate it. Accepted; noted for completeness.

Test

`test_L2E_OracleSurvivesTheUint32EpochRollover` builds ring history, warps past 2^{32} , and asserts `poke()`, `markSqrtPriceX96()` and `twapTick()` all survive and keep producing a mark afterwards.

5.11 F-11, F-12 — L2 timing findings (documented, not fixed)

F-11	Medium	Anti-snipe surtax is quantised to parent blocks	Status: documented
-------------	---------------	--	---------------------------

CauldronHook.sol:421-435, 1040-1068

The surtax window and its decay are counted in `block.number` (`snipeWindowBlocks = 30, elapsed = block.number - start`). On the target that is the parent chain's number, so two things happen at once.

The good half. The window keeps roughly its intended *wall-clock* length: 30 parent blocks is about six minutes, the same as on L1. This is genuinely lucky — unlike a block-denominated *death* clock, which would have been catastrophically wrong, a parent-block-denominated *launch* window happens to carry over.

The bad half. Its *resolution* collapses. Every child block inside one parent block sees identical `elapsed`, and `blockhash(block.number - 1)` — one of the two jitter inputs — is likewise constant for that entire ~ 12 seconds. At a 250 ms cadence that is roughly 48 child blocks at one price. A sniper faces a step function, not a ramp, and can place a buy anywhere inside a step at identical cost.

The only input that still varies within a step is the pool's live `tick (:1062)`, which was added precisely to be unpredictable at submission time. That mitigation survives the L2 transition and is the reason this is Medium rather than High. But note the composition with F-01: the jitter's other input is a `blockhash`, so if that is predictable the jitter reduces to the tick alone.

Evidence. `test_L2B_SurtaxIsQuantisedToParentBlocks` holds `block.number` fixed, advances 11 seconds, and asserts the rate is *unchanged*; then advances one block and shows it decays. `testFuzz_L2B_SurtaxStaysWithinItsCeiling` fuzzes independent block and time jumps and confirms the rate never exceeds `MAX_SNIPE_BPS`.

Remediation. Denominate the window in seconds, as the volume clock already is. That is a small change to `_defaultSurtaxBps` plus renaming `snipeWindowBlocks`, but it costs hook bytecode (F-14) and changes a governance-visible parameter's meaning. Given the window's wall-clock length is approximately preserved, the pragmatic path is: size `snipeWindowBlocks` in *parent* blocks deliberately ($30 \approx 6$ min) and document that the decay is a 12-second step function — and accept quantisation as the cost.

F-12	Medium	MIN_TWAP of 1 s is not a meaningful floor on a fast L2	Status: documented
-------------	---------------	---	---------------------------

PerpEngine.sol:185, 1263, 1275-1278

`MIN_TWAP = 1 seconds` is justified in-code as “fast L2: sub-second blocks $\rightarrow$ even 1s spans many blocks”, and `setRisk` enforces `warmup >= MIN_TWAP` with the comment that this is what makes the liquidation mark manipulation-resistant.

The reasoning inverts. At 250 ms blocks a one-second average spans about *four* blocks. Under first-come-first-served sequencing with no priority auction, landing a handful of consecutive transactions is a matter of submission timing, not fee bidding — so a four-block average is a weak defence, not a strong one. The claim that the warmup floor guarantees a manipulation-resistant mark does not hold at the configurable floor.

In practice the effective window is longer than `twapWindow` suggests, because ring appends are throttled to `OBS_INTERVAL = 15 seconds (:404-408)`: the binary search lands on an observation between `twapWindow` and `twapWindow + 15 s` old, so even `twapWindow = 1` yields a 1–16 second average. That is a real mitigation and the reason this is Medium. It is also *accidental* — it comes from a gas optimisation, not from a stated safety bound.

Remediation (configuration, no code change).

1. Do not set `twapWindow` anywhere near `MIN_TWAP` on the target. Choose it from the pool's realistic depth: a window long enough that moving the average costs more than the maximum

liquidation payout it could trigger.

2. Set `warmup` independently and generously — it exists to cover the oracle’s cold start, and tying its floor to `MIN_TWAP` conflates two unrelated concerns.
3. Raise `MIN_TWAP` to `OBS_INTERVAL` at the next engine deploy, so the floor reflects the ring’s actual resolution rather than a value no configuration can honour.

5.12 F-13 to F-17 — lower-severity and informational

F-14 Low EIP-170 headroom is nearly exhausted Status: documented

Post-remediation deployed sizes against the 24,576-byte limit:

Contract	Bytes	Headroom
CauldronHook	24,488	88 <0.4%
PerpEngine	24,353	223 0.9%
CauldronRegistry	24,237	339 1.4%
MiFrensGenesis	18,593	5,983 ample

This is a *process* risk, and it is why F-01 ships without a code fix. The hook cannot absorb a meaningful change — including an emergency one. (The engine was at 48 bytes before this audit; the unchecked blocks that fix F-10 removed overflow checks and returned it to 223, which is a reminder of how thin the margin is: one arithmetic guard either way moves it by 175 bytes.) Note also that these sizes depend on `optimizer_runs = 1` and `via_ir = true` in the cauldron profile: the build configuration is load-bearing for deployability, and a routine compiler or optimiser-setting change could push either contract over the limit.

Recommendation. Before the next feature lands, reclaim headroom by extraction rather than deletion. `CauldronVault` integration is the obvious candidate (see F-17). Add a CI assertion that fails the build below a floor of, say, 1,000 bytes on both contracts, so the constraint is enforced rather than rediscovered.

F-16 Low enableAutoMigrate keeps overpayment Status: documented

`CauldronRegistry.sol:1027-1037`

The check is `if (msg.value < AUTO_MIGRATE_FEE) revert Fee();` with no refund of the excess. A user sending a round 0.1 ETH silently donates 0.031 ETH. The funds are not lost to the protocol (they seed the next launch), but the asymmetry is unnecessary. **Recommendation:** refund the surplus, or require exact payment. Not fixed here because it is a behaviour change to a live endpoint with no security consequence, and registry headroom is better spent elsewhere.

F-13 Info The documented layout-verification command does not run Status: documented

`CauldronBase.sol:76-79`

`CauldronBase` instructs the reader to verify the `delegatecall` pair with `forge inspect CauldronRegistry storageLayout` and confirm the facet's output is byte-identical. As configured, that command fails: `Error: storage layout missing from artifact`. The cauldron profile does not request `storageLayout` in its build output.

This matters because the instruction guards the single most dangerous property in the system, and a verification step that errors out is one people stop running. **Recommendation:** add `extra_output = ["storageLayout"]` to the cauldron profile. (This audit added it temporarily to perform the check — see Appendix A — and reverted it, to keep the diff limited to security changes.)

F-15 Info Funding overflow at an extreme mark Confidence: Suspected

`PerpEngine.sol:513-520`

`_quoteMark(shortOiToken)` scales as $1/\text{sqrtPrice}^2$. Near `MIN_TICK` the mark value of the short OI approaches $\sim 3.4 \times 10^{38}$ per unit of size, and `imbalance * rate * dt * 1e18` would then overflow `int256` for modest `dt`, reverting `_pokeFunding` — the same brick chain as F-04 and F-10. Reaching that tick requires trading through the entire active band and the reserve ceiling while a large short

OI is open, which the `max0iBps` and `maxNotionalBps` caps make implausible. Labelled **Suspected** and recorded for completeness; the F-04 rate bound already removes the largest multiplier.

F-17 Info CauldronVault.redeem is permanently unreachable

CauldronVault.sol:94-112, CauldronRegistry.sol:898, 922

Both collection-deployment paths call `hook.setVault(address(0))`, so no ETH ever reaches a `CauldronVault` and `redeem` can never succeed. The contract handles this gracefully — it reverts `UnifiedFloorActive()` to point holders at the token-denominated floor instead — and the vault is still load-bearing, because `crystallizeCollection` reads `outstanding()` to size the entitlement at death.

Recorded not as a bug but as the clearest opportunity for F-14: a vestigial redemption path is dead weight in a system that is 48 bytes from its limit.

6 L2 deployment considerations (Arbitrum Nitro / Orbit)

Every EVM primitive whose semantics differ on the target was audited against its uses in the corpus. The table records the documented behaviour, the protocol’s exposure, and the verdict.

Primitive	Documented behaviour	target	be-	Exposure in scope	Verdict
block.number	Estimate of the <i>parent chain’s</i> block number; “updates only periodically”			Surtax window Hook:421,1046; gacha commit :1694,1731; rarity commit Collection:203, Genesis:405; governor snapshot Gov:179,217; CauldronToken.birthBlock	F-11 quantisation; else safe
block.timestamp	Set per child block from the sequencer’s clock; sequencer may place it up to 24h early / 1h late; blocks produced only when there are transactions			Volume window Hook:1139,1165; minLifetime; TWAP ring; funding dt; liq throttle; vesting; seed schedule	Correct choice (+F-10 fixed)
blockhash(n)	“Cryptographically insecure, pseudo-random”; “do not come from L1”; “should not be relied on as a secure source of randomness”			Gacha roll Hook:1732; rarity reveal Collection:215, Genesis:417; surtax jitter Hook:1064	F-01
prevrandao / difficulty	“Returns the constant 1”			Zero uses in the entire corpus	Not affected
gasleft() / 63-64	Standard EVM; L2 gas plus an L1 calldata component, so absolute costs differ			LIQ_, GACHA_, LEGACY_, SEED_POKE_ reserves; RELAUNCH_TAIL_RESERVE; badge mint	Re-tune, see below
tx.origin	Standard for direct child-chain transactions; aliased for parent-chain-initiated messages			Crystal credit Hook:634; liquidation keeper :704	Accepted; see below
Sequencer	Single sequencer, “first-come, first-served ... based on sequencer arrival time”; no priority gas auction			Anti-snipe; TWAP manipulation; relaunch races	F-11, F-12
block.coinbase	Returns a designated internal sequencer address			Zero uses	Not affected

6.1 Gas reserves

The protocol makes heavy, correct use of the “reserve gas, forward the rest, ignore the result” pattern so that an optional side-effect can never revert or starve a user’s swap. Six such reserves exist, from LIQ_GAS_RESERVE = 180,000 up to RELAUNCH_TAIL_RESERVE = 8,000,000.

The pattern itself is sound on any EVM chain and the values are conservative. What changes on the target is the *denominator*: transaction cost has an L1 calldata component that L2 execution gas does not capture, and gas estimation behaves differently as a result. The consequence is not a safety failure — a too-small reserve makes an optional step no-op, which every call site already handles — but a *liveness* one: if the reserves are mis-sized relative to real child-chain block limits, the in-swap liquidation sweep, the seeder poke and the legacy buyback could silently never fire, quietly turning a keeperless design back into one that needs keepers.

Recommendation. On the first testnet deployment, instrument how often each gas-bounded call actually executes versus no-ops. The seeder poke is the one to watch: `SEED_POKE_GAS_MIN = 750,000` is a high bar for an ordinary swap, and if it is rarely met the progressive stream depends entirely on the permissionless `poke()` fallback — which works, but is a different operational model than the one documented.

6.2 tx.origin attribution

`tx.origin` is used deliberately in two places and the reasoning at `CauldronHook.sol:620-630` is correct: crediting `sender` would send crystals to the router rather than to the person who paid. On the target, `tx.origin` behaves normally for direct transactions. The known limitation — it is the bundler for ERC-4337 accounts — is already documented and mitigated by `creditUntaggedSwaps` plus the tagging-opener path. No change recommended; recorded so the L2 review is complete.

6.3 Reorgs and finality

Under a single sequencer, child-chain transactions are soft-confirmed on arrival and only become parent-chain-final later. Nothing in the corpus assumes cross-chain finality, and no path reads parent-chain state. The relevant exposure is that a *soft-confirmed* relaunch or gacha resolution could in principle be re-ordered before finality. Because the relaunch is idempotent-by-guard (`isDead` plus `minLifetime` plus `markConsumed`) and the gacha is FIFO with the seed pinned to a block, a reorg re-executes rather than corrupts. No finding.

7 Attacks constructed and refuted

A refuted attack is evidence. Each of these was worked through to the point where it either had to succeed or be shown impossible.

Squatting the next generation's token address

Attack. If the token were deployed with `CREATE2` keyed on the generation number, every input would be public before the deploying transaction: the salt is the next generation, the deployer is the registry, and `(name, symbol)` come from `governor.winner()`, a public view. Occupying that address would make `new` revert inside `relaunch()`, rolling back `markConsumed` — so the same proposal wins forever and the machine is permanently bricked.

Refuted. `PoolOps.sol:432-437` uses plain `CREATE`. The address derives from `(registry, registry nonce)`, and only the registry can advance its own nonce. Occupying it needs a 2^{160} preimage search. *Structurally immune.*

Draining the reserve through the delegatecall facet

Attack. Point `redemptionExt` at malicious code, or exploit a layout mismatch so the facet writes the wrong slots, or re-enter the forwarder.

Refuted, three ways. (i) `setRedemptionExt (CauldronRegistry.sol:166-174)` is `onlyOwner`, one-shot (if `(redemptionExt != address(0)) revert`), and rejects code-less targets — which matters, because a delegatecall to an empty account returns *success* with no data, making every redemption a silent no-op. (ii) The layouts are byte-identical, verified mechanically (Appendix A). (iii) The forwarders correctly omit `nonReentrant` while the facet functions keep it, and OpenZeppelin 5.6's guard uses a namespaced ERC-7201 slot shared under delegatecall — so re-entry is blocked without double-locking. Calling a deployed `RedemptionExt` directly is harmless: it runs against its own empty storage where `summoned` is false and `positionManager` is zero.

Farming the gacha pity counter

Attack. `missStreak >= pityThreshold` forces a win. Deliberately open crystals at near-zero odds to bank cheap misses, then collect a guaranteed creature.

Refuted by arithmetic. Crystals cost *credit* along the rising curve (`CauldronHook.sol:1672-1677`) regardless of the odds attached to them. With `pityThreshold = 8`, a zero-odds player yields one creature per nine crystals; a 90%-odds player yields one per ~ 1.11 . High odds dominate at every point. The strategy is strictly worse than playing honestly.

Fee bypass through the exact-output sell quadrant

Attack. Of the four swap quadrants, an exact-output sell has the token as its unspecified currency, so `v4's afterSwap` return delta physically cannot take an ETH fee there — and `beforeSwap` skips it for not being exact-input.

Refuted — already closed. `CauldronHook.sol:887` reverts `ExactOutSellUnsupported` for exactly that quadrant. Refusing the quadrant rather than serving it free is the right call for an ETH-fee-only hook: the exact-input sell is economically equivalent and unaffected. Verified by reading all four quadrants: exact-in buy charged in `beforeSwap`; exact-in sell and exact-out buy charged in `afterSwap`; exact-out sell refused.

Leaving a v4 delta unsettled through nested composition

Attack. One user swap can reach a nested legacy-buyback swap, a seeder `modifyLiquidity`, perp liquidation settlement swaps and an NFT mint, all inside one unlock. Find a composition where

some contract's delta goes unsettled, so the user's swap reverts (grief) or the pool is left short (theft).

Refuted. Each participant settles its own deltas within its own frame — see invariant V2 for the four call sites. Two details make the composition safe rather than merely lucky: `legacyBuyStep` settles the *realised* debit rather than the intended amount, so a binding price limit cannot leave a positive un-taken delta; and `PerpEngine._run` switches to an in-place swap when `_inLocked`, because re-entering `unlock` inside a hook callback would revert. Both are correct and both are non-obvious.

Poisoning the perp mark with a stale tick

Attack. Crash spot with a large sell, restore it in the same transaction, and leave `lastTick` frozen at the crashed value; `twapTick` then extrapolates the un-recorded tail at that tick, so solvent positions read liquidatable.

Refuted — already closed. `_writeObs` (`PerpEngine.sol:395-410`) always refreshes `lastTick` and always integrates the elapsed interval, with ring *appends* throttled on a separate clock (`lastRingTs`) so flood-resistance is preserved. The related empty-book variant is also closed: `_doSweep` pokes the oracle *before* its empty-book early return (`:690-692`), so a quiet period cannot freeze the tick. Both fixes are correct and their comments explain the attack accurately.

Growing the perp book beyond what one relaunch can clear

Attack. Open more positions than `forceCloseAllDead` clears per call; the survivors outlive the rebirth, `syncGeneration` reverts `PositionsOpen` forever, and the token inventory is stranded in a dead generation — where those positions can no longer be closed at all, since `_settle` would swap old-generation sizes against the new pool.

Refuted structurally. `MAX_OPEN_POSITIONS = 64` is enforced in `_book (:1018)` and sits strictly below `FORCE_CLOSE_MAX = 96`, so one call always drains the book. Both are *constant*, not tunable — correctly, since they underwrite a structural guarantee. A `minCollateral` dust filter blocks the spam variant.

Freezing settlement with a hostile recipient

Attack. Open a position from a contract whose `receive()` reverts. Settlement can then never pay out, so the position can never be closed — cascading into the `forceCloseAllDead` / `syncGeneration` brick above.

Refuted — already closed. `_payOut` (`PerpEngine.sol:1085-1089`) credits `payoutOwed` on failure instead of reverting, with a bounded 30k forwarding budget so the recipient cannot consume the settlement's gas either. Funds stay claimable via `claimPayout`. The same pattern is applied to the `Liquidator` badge, including the subtle case that a low-level call to a code-less address returns `true` — guarded at `:1144`.

Stealing the hook's ETH by re-pointing the registry

Attack. The hook owner re-points `registry` to an address they control and calls `releaseRelaunchETH()`.

Refuted. `setRegistry` is one-shot (`CauldronHook.sol:1337-1341`). The upgrade path (`proposeRegistryOverride` / `executeRegistryOverride`) requires a 7-day announced delay via a *constant* the owner cannot shorten, and *flushes* `relaunchETH` to the outgoing registry first (`:1377-1383`), so a controller swap cannot capture ETH the previous registry accrued.

Serving an attacker-created pool

Attack. v4 lets anyone initialise a pool naming any hook. Create one without an ETH leg, or one priced by the attacker, so the hook mints phantom `relaunchETH` or aims its legacy buyback at a controlled book.

Refuted, twice over. `_afterInitialize` adopts a pool only if `sender == registry` *and* `currency0` is native ETH (:534-537), and declines rather than reverts — so pool creation cannot be grieved. Every value-moving path is then gated on `_served`. Separately, the legacy buyback spends only into `_liveKey`, the key the registry recorded, never the caller's key (:763-776). Crystal credit is additionally gated on the swap's `currency1` matching the live key (:614-617), so a retired generation's pool cannot farm the newborn collection.

8 Test suite added by this audit

27 tests across four new files under `test/final/`. No existing test file was modified.

Test	Finding	Asserts
F01_CustodyAndConsent.t.sol — no fork required, so these are real CI gates		
<code>test_F02_AutoMigrateConsentIsRevocable</code>	F-02	consent can be withdrawn; re-opting still costs the fee
<code>test_F02_RevocationIsSelfServiceOnly</code>	F-02	one wallet cannot revoke another's
<code>test_F03_PartialSweepKeeps...</code>	F-03	the un-transferred remainder stays owed and is later recoverable
<code>testFuzz_F03_SweepDebitsExactly...</code>	F-03	$\forall(\text{owed}, \text{bal}): \text{moved} = \min, \text{counter debited by exactly that}$
<code>test_F03_SweepIsRegistryGated</code>	F-03	access control
<code>test_F09_GasStarvedTransferIsRefused</code>	F-09	the measured window's midpoint no longer settles
<code>test_F09_TheEntireGriefWindowIsClosed</code>	F-09	110k–132k in 2k steps: a settled transfer always breaks the spell
<code>test_F09_NormalTransferStill...</code>	F-09	control: ordinary transfers unaffected
<code>test_F09_RevertingDividend...</code>	F-09	control: a hostile dividend still cannot brick a transfer
F02_L2Semantics.t.sol — fork; advances <code>vm.roll</code> and <code>vm.warp</code> independently		
<code>test_L2A_DeathClockIsWallClock...</code>	L1	50k blocks with time frozen kills nothing; a day of clock kills on schedule
<code>test_L2A_BlockStormAloneCannot...</code>	L1	100k block ticks cannot retire a pool with live volume
<code>test_L2B_SurtaxIsQuantised...</code>	F-11	no decay without a block tick; window still closes
<code>testFuzz_L2B_SurtaxStaysWithin...</code>	F-11	$\forall$ independent jumps: $\text{rate} \leq \text{MAX_SNIPE_BPS}$
<code>test_L2C_TicketMaturityFollows...</code>	F-01	time alone does not mature a ticket; a block tick does
<code>test_L2C_RollIsFullyDetermined...</code>	F-01	the roll carries no entropy beyond the blockhash
<code>test_L2D_FundingRateIsBounded</code>	F-04	rejects 2^{255} , $\text{max}, 10^4+1$; accepts the ceiling and the default
<code>testFuzz_L2D_PokeSurvives...</code>	F-04	$\forall$ accepted rate $\times$ a year of decoupled clock: poke lives
<code>test_L2E_OracleSurvivesTheUint32...</code>	F-10	the oracle survives the 2106 rollover and still produces a mark
F04_ReserveCeiling.t.sol — fork; drives a real 69 $\times$ pump on live v4		
<code>test_F18_MigrationWorksBelow...</code>	F-18	baseline: both exits work below the ceiling
<code>test_F18_MigrationBreaksAbove...</code>	F-18	above it: <code>claimByBurn</code> and <code>redeemOgFren</code> both revert; balances intact
<code>test_F18_CeilingIsSetBefore...</code>	F-18	<code>setReserveCeiling</code> cannot move a live generation's band
F03_FinalInvariants.t.sol — handler-driven, 128,000 calls per invariant		
<code>invariant_L1_NoOwedValueIsEver...</code>	F-03	swept + owed = staged, always

Test	Finding	Asserts
invariant_L2_SweptNeverExceeds...	R3	a sweep never over-delivers
invariant_L3_HookHoldsWhatItOwes	R3	an over-statement stays visible, never erased
invariant_C1_ConsentMatchesThe...	F-02	the flag always equals that actor's own last choice
invariant_S1_ScheduleIsMonotone...	R4	the stream target never decreases and never exceeds 100%
testFuzz_S1_ScheduleIsTotal	R4	the schedule is total, bounded and monotone for all inputs

8.1 Reproduction

```
cd contracts/solidity

# 1. Off-chain suites (no RPC needed) -- 15 tests
FOUNDRY_PROFILE=cauldron forge test --match-path 'test/final/F01*'
FOUNDRY_PROFILE=cauldron forge test --match-path 'test/final/F03*'

# 2. Fork suites -- 12 tests
export FORK_RPC=https://ethereum-sepolia-rpc.publicnode.com \
    POOL_MANAGER=0xE03A1074c86CFedD5C142C4F04F1a1536e203543 \
    POSITION_MANAGER=0x429ba70129df741B2Ca2a85BC3A2a3328e5c09b4
FOUNDRY_PROFILE=cauldron forge test --match-path 'test/final/F02*' -vv
FOUNDRY_PROFILE=cauldron forge test --match-path 'test/final/F04*' -vv

# 3. Everything, including the pre-existing suites
FOUNDRY_PROFILE=cauldron forge test
```

Suite	Tests	Result
Pre-existing	312	pass no regressions from the nine fixes
Added by this audit	27	pass each verified to fail pre-fix where applicable
Total	339	339 pass, 0 fail 66 suites, 108s wall

Note on the fork RPC

The POOL_MANAGER address given in the engagement brief was truncated. The correct Sepolia v4 PoolManager for this repository is 0xE03A1074c86CFedD5C142C4F04F1a1536e203543, taken from `.env.sepolia`. With the truncated value every fork suite fails in `setUp` with “*call to non-contract address*”, which is easy to mistake for a broken test rather than a bad environment.

9 Pre-deployment checklist

Ordered by consequence. Items 1–3 should block launch.

1. **Choose the reserve ceiling deliberately for every iteration** (F-18), and decide now whether to build the governance-gated reserve re-placement. Shipping with the $69\times$ default un-revisited means the exit guarantee has an undisclosed price cap. At minimum, qualify the guarantee wherever it is stated.
2. **Resolve F-01 before enabling the gacha.** Deploy the `blockhash` probe described in F-01 on the target and determine whether the sequence is predictable. If it is, the gacha and the rarity reveal must not go live in their current form. This is the one open High.
3. **Set `twapWindow` from real pool depth, not from `MIN_TWAP`** (F-12). Choose a window long enough that moving the average costs more than the largest liquidation it could trigger. Set warmup separately and generously.
4. **Set a non-zero `EMERGENCY_DELAY`** (F-19). The structural hole is fixed — arming is now mandatory, so the forced-open exit engages at any delay — but the delay is still what gives holders a real *window* rather than a transaction boundary. The live round-31 registry was constructed with `emergencyDelay = 0`; the value is `immutable`, so only the planned redeploy corrects it. The deploy script's 48 h default is right: verify the constructor arguments on the *deployed bytecode*, not the script, and confirm `owner()` is the timelock rather than the presale (it currently is not).
5. **Re-tune the six gas reserves against real child-chain limits** and instrument how often each gas-bounded call fires versus no-ops. Watch `SEED_POKE_GAS_MIN = 750,000` in particular: if ordinary swaps rarely clear it, the progressive stream is keeper-dependent in practice.
6. **Add `extra_output = ["storageLayout"]`** to the cauldron profile (F-13) and re-run the registry/facet layout comparison as a CI step, not a manual one.
7. **Add an EIP-170 headroom assertion to CI** (F-14). At 48 and 88 bytes, `PerpEngine` and `CauldronHook` can be pushed over the limit by a compiler bump alone.
8. **Size `snipeWindowBlocks` in parent blocks deliberately** (F-11) and document the decay as a 12-second step function, so operators are not reasoning about a smooth ramp that does not exist.
9. **Confirm `setReserveCeiling` per iteration.** The $69\times$ default is a per-iteration choice with real consequences: the reserve only backs redemption while spot stays above the band.
10. **Decide the shortfall policy** for the two branches F-06 makes observable. `ReserveShortfall` now fires in both cases; someone has to be watching it, and there has to be an agreed response (proportional haircut versus first-come).
11. **Monitor `ReserveShortfall`, `BadDebt`, `PayoutOwed` and `UnattributedYield`.** All four are the protocol correctly reporting a state that needs a human decision.

A Registry / facet storage layout verification

Produced by compiling with `extra_output = ["storageLayout"]` and comparing `out/CauldronRegistry.sol/CauldronRegistry.json` against `out/RedemptionExt.sol/RedemptionExt.json` on (label, slot, offset, type).

Result: 53 entries each — identical in every field.

Slot	Offset	Variable
0	0	<code>_owner</code> (<i>Ownable</i>)
0	20	<code>nextReserveCeilingOffset</code>
0	23	<code>summoned</code>
0	24	<code>_seedBuyUnlocked</code>
1	0	<code>currentGeneration</code>
2	0	<code>currentToken</code>
3–11	0	<code>generation maps: Token, Proposer, Parent, PoolId, PoolKey, PositionId, ReservePositionId, reserveTickLower, reserveTickUpper</code>
12	0	<code>genesisReserveOutstanding</code>
13–15	0	<code>claimed, generationCollection, generationVault</code>
16–17	0	<code>collectionLedger, genesisPending</code>
18–20	0	<code>factory, governor, nftMaxSupply</code>
21	0 / 20	<code>royaltyDividend royaltyBps</code> (<i>packed</i>)
22–24	0	<code>genesisMode, genesisBaseURI, genesisRenderer</code>
25–28	0	<code>mifrens, genesisBonusBps, genesisShares, genesisSharePerFren</code>
29	0	<code>enchantFeeMultBps</code>
30	0 / 1	<code>redemptionPaused guardian</code> (<i>packed</i>)
31–32	0	<code>successor, claimGate</code>
33–36	0	<code>airdropWallet, airdropReserve, primeBuyEth, primeFunder</code>
37–39	0	<code>emergencyReadyAt, lastSummonAt, minLifetime</code>
40	0	<code>autoMigrate</code> (<i>end of the documented baseline</i>)
41	0	<code>poolManager</code> (<i>was immutable</i>)
42	0	<code>positionManager</code> (<i>was immutable</i>)
43	0	<code>hook</code> (<i>was immutable</i>)
44	0	<code>redemptionExt</code>
45	0 / 20	<code>seeder nextSeedWindow</code> (<i>packed</i>)
46	0	<code>igniter</code>

Two observations.

The promotion of `poolManager`, `positionManager` and `hook` from *immutable* to storage is correct and is the kind of thing that is easy to get wrong. *Immutable*s are baked into the *executing* contract's code; under *delegatecall* the facet executes its own code, where those values were never set, so they would read as zero — and a zero `positionManager` means every reserve operation calls into nothing.

ReentrancyGuard's `_status` appears in neither layout because OpenZeppelin 5.6 stores it at an ERC-7201 namespaced slot, a compile-time constant identical in both contracts. This is what makes the shared guard work across the delegatecall, and it is why the registry's forwarders must not add their own `nonReentrant` — doing so would double-lock and revert every redemption.

B Complete function inventory

735 external and public ABI entries across the 23 in-scope contracts and libraries, extracted from the compiled artifacts. Internal and private functions bring the in-file declaration count to approximately 550.

CauldronRegistry (108 external/public entries)

Signature	Mutability
<code>AUTO_MIGRATE_FEE()</code>	view
<code>GEN1_ACTIVE_TOKENS()</code>	view
<code>MAX_NFT_SUPPLY()</code>	view
<code>POOL_FEE()</code>	view
<code>TICK_SPACING()</code>	view
<code>TOTAL_SUPPLY()</code>	view
<code>airdropReserve()</code>	view
<code>airdropWallet()</code>	view
<code>armEmergency()</code>	nonpayable
<code>autoMigrate(address)</code>	view
<code>autoMigrateBatch(uint256,address[])</code>	nonpayable
<code>buyCollectionNFT(uint256,uint256)</code>	nonpayable
<code>buyTreasury0gFren(uint256)</code>	nonpayable
<code>claimByBurn(uint256,uint256)</code>	nonpayable
<code>claimByBurnUpTo(uint256,uint256)</code>	nonpayable
<code>claimGate()</code>	view
<code>claimed(uint256,address)</code>	view
<code>collectionLedger()</code>	view
<code>constructor(address,address,address,address,uint256)</code>	nonpayable
<code>currentGeneration()</code>	view
<code>currentToken()</code>	view
<code>disableAutoMigrate()</code>	nonpayable
<code>donateToReserve(uint256)</code>	nonpayable
<code>emergencyAdmin()</code>	view
<code>emergencyDelay()</code>	view
<code>emergencyReadyAt()</code>	view
<code>emergencySweep(address)</code>	nonpayable
<code>emergencyWithdrawLP(uint256)</code>	nonpayable
<code>enableAutoMigrate()</code>	payable
<code>enchantFee()</code>	view
<code>enchantFeeMultBps()</code>	view
<code>factory()</code>	view
<code>floorClaimableNow()</code>	view
<code>floorPerFren()</code>	view
<code>fundPrimeBuy()</code>	payable
<code>generationCollection(uint256)</code>	view

Signature	Mutability
generationParent(uint256)	view
generationPoolId(uint256)	view
generationPoolKey(uint256)	view
generationPositionId(uint256)	view
generationProposer(uint256)	view
generationReservePositionId(uint256)	view
generationToken(uint256)	view
generationVault(uint256)	view
genesisBaseURI()	view
genesisBonusBps()	view
genesisMode()	view
genesisRenderer()	view
genesisReserveOutstanding()	view
genesisSharePerFren()	view
genesisShares()	view
governor()	view
guardian()	view
hasClaimed(uint256,address)	view
hook()	view
igniter()	view
lastSummonAt()	view
materializeLegacyReserve()	nonpayable
mifrens()	view
migrateToSuccessor()	nonpayable
minLifetime()	view
nextSeedWindow()	view
nftMaxSupply()	view
owner()	view
poolManager()	view
positionManager()	view
primeBuyEth()	view
primeFunder()	view
receive()	payable
recycleCollectionNFT(uint256,uint256)	nonpayable
redeemOgFren(uint256)	nonpayable
redemptionExt()	view
redemptionPaused()	view
relaunch()	nonpayable
renounceOwnership()	nonpayable
rescueSeeder()	nonpayable
reserveTickLower(uint256)	view
reserveTickUpper(uint256)	view
royaltyBps()	view
royaltyDividend()	view
seeder()	view
setAirdropReserve(address,uint256)	nonpayable
setClaimGate(address)	nonpayable
setCollectionLedger(address)	nonpayable
setEnchantFeeMult(uint256)	nonpayable
setFactory(address)	nonpayable

Signature	Mutability
setGenesisBonus(address,uint256,uint256)	nonpayable
setGenesisMetadata(uint8,string,address)	nonpayable
setGovernor(address)	nonpayable
setGuardian(address)	nonpayable
setIgniter(address)	nonpayable
setMinLifetime(uint256)	nonpayable
setNftMaxSupply(uint256)	nonpayable
setPrimeFunder(address)	nonpayable
setRedemptionExt(address)	nonpayable
setRedemptionPaused(bool)	nonpayable
setReserveCeiling(int24)	nonpayable
setRoyalty(address,uint96)	nonpayable
setSeedWindow(uint64)	nonpayable
setSeeder(address)	nonpayable
setSuccessor(address)	nonpayable
successor()	view
summon()	payable
summoned()	view
sweepPrimeBuy()	nonpayable
transferOwnership(address)	nonpayable
unlockCallback(bytes)	nonpayable
vetoEmergency()	nonpayable

CauldronHook (137 external/public entries)

Signature	Mutability
BPS()	view
HOURS_PER_DAY()	view
MAX_MINTS_PER_CALL()	view
MAX_PROPOSER_BPS()	view
MAX_SNIPE_BPS()	view
MAX_TAX_BPS()	view
MAX_TOTAL_FEE_BPS()	view
MAX_WEIGHT_BPS()	view
ODDS_HARD_CAP_BPS()	view
REGISTRY_SWAP_DELAY()	view
activeProposer()	view
afterAddLiquidity(address,tuple,tuple,int256,int256,bytes)	nonpayable
afterDonate(address,tuple,uint256,uint256,bytes)	nonpayable
afterInitialize(address,tuple,uint160,int24)	nonpayable
afterRemoveLiquidity(address,tuple,tuple,int256,int256,bytes)	nonpayable
afterSwap(address,tuple,tuple,int256,bytes)	nonpayable
batchCursor()	view
batches(uint256)	view
beforeAddLiquidity(address,tuple,tuple,bytes)	nonpayable
beforeDonate(address,tuple,uint256,uint256,bytes)	nonpayable
beforeInitialize(address,tuple,uint160)	nonpayable
beforeRemoveLiquidity(address,tuple,tuple,bytes)	nonpayable

Signature	Mutability
<code>beforeSwap(address,tuple,tuple,bytes)</code>	nonpayable
<code>buyWeightBps()</code>	view
<code>cancelRegistryOverride()</code>	nonpayable
<code>claimProposerFees()</code>	nonpayable
<code>collection()</code>	view
<code>commitCrystals(address,uint256,uint256)</code>	nonpayable
<code>committedOf(address)</code>	view
<code>constructor(address,uint256,address,address,address)</code>	nonpayable
<code>costOfNextCrystals(uint256)</code>	view
<code>creditEpoch()</code>	view
<code>creditOf(address)</code>	view
<code>creditUntaggedSwaps()</code>	view
<code>crystalsReady(address)</code>	view
<code>cumulativeVolume()</code>	view
<code>curvePolicy()</code>	view
<code>deathChecker()</code>	view
<code>deathThreshold()</code>	view
<code>defaultTaxBps()</code>	view
<code>executeRegistryOverride()</code>	nonpayable
<code>feeRouter()</code>	view
<code>floorBps()</code>	view
<code>forceClosePerps()</code>	nonpayable
<code>fundLegacyBuffer()</code>	payable
<code>getHookPermissions()</code>	pure
<code>getVolume24h(bytes32)</code>	view
<code>guild()</code>	view
<code>guildBps()</code>	view
<code>isDead(bytes32)</code>	view
<code>isOpenener(address)</code>	view
<code>legacyBps()</code>	view
<code>legacyBuffer()</code>	view
<code>legacyBuyStep(tuple)</code>	nonpayable
<code>legacyOwedToReserve()</code>	view
<code>legacyRegistry()</code>	view
<code>legacyThreshold()</code>	view
<code>lifetimeVolumeOf(address)</code>	view
<code>liveKey()</code>	view
<code>maxOddsBps()</code>	view
<code>mintBaseline()</code>	view
<code>mintedOut()</code>	view
<code>missStreak(address)</code>	view
<code>nativeGachaStep(address,uint256)</code>	nonpayable
<code>nftContract()</code>	view
<code>nftCredit(uint256,address)</code>	view
<code>nftPriceAt(uint256)</code>	view
<code>nftPriceStep()</code>	view
<code>oddsForPlay(uint256)</code>	view
<code>oddsFullVolumeWei()</code>	view
<code>oddsPolicy()</code>	view
<code>opened(address)</code>	view

Signature	Mutability
<code>outstandingCrystals()</code>	view
<code>outstandingOf(address)</code>	view
<code>outstandingTickets()</code>	view
<code>owner()</code>	view
<code>pendingOf(address)</code>	view
<code>pendingRegistry()</code>	view
<code>perpEngine()</code>	view
<code>pityThreshold()</code>	view
<code>poolInitBlock(bytes32)</code>	view
<code>poolManager()</code>	view
<code>progress(address)</code>	view
<code>proposeRegistryOverride(address)</code>	nonpayable
<code>proposerBps()</code>	view
<code>proposerOwed(address)</code>	view
<code>quest()</code>	view
<code>receive()</code>	payable
<code>registry()</code>	view
<code>registrySwapReadyAt()</code>	view
<code>relaunchETH()</code>	view
<code>releaseRelaunchETH()</code>	nonpayable
<code>renounceOwnership()</code>	nonpayable
<code>resolveTickets(uint256)</code>	nonpayable
<code>seeder()</code>	view
<code>sellWeightBps()</code>	view
<code>setActiveProposer(address)</code>	nonpayable
<code>setCollection(address)</code>	nonpayable
<code>setCreditUntaggedSwaps(bool)</code>	nonpayable
<code>setDeathChecker(address)</code>	nonpayable
<code>setDeathThreshold(uint256)</code>	nonpayable
<code>setDefaultTaxBps(uint256)</code>	nonpayable
<code>setFeeRouter(address)</code>	nonpayable
<code>setFloorBps(uint256)</code>	nonpayable
<code>setGuild(address)</code>	nonpayable
<code>setGuildBps(uint256)</code>	nonpayable
<code>setLegacyBuyback(address,uint256,uint256)</code>	nonpayable
<code>setLiveKey(tuple)</code>	nonpayable
<code>setMaxOdds(uint256)</code>	nonpayable
<code>setNftContract(address)</code>	nonpayable
<code>setNftCurve(uint256,uint256)</code>	nonpayable
<code>setNftCurveFrom(uint256)</code>	nonpayable
<code>setOddsParams(uint256,uint256)</code>	nonpayable
<code>setOpener(address,bool)</code>	nonpayable
<code>setPerpEngine(address)</code>	nonpayable
<code>setPolicies(address,address,address)</code>	nonpayable
<code>setProposerBps(uint256)</code>	nonpayable
<code>setQuest(address)</code>	nonpayable
<code>setRegistry(address)</code>	nonpayable
<code>setSeeder(address)</code>	nonpayable
<code>setSnipeParams(uint256,uint256)</code>	nonpayable
<code>setTaxExempt(address,bool)</code>	nonpayable

Signature	Mutability
setTreasury(address)	nonpayable
setVault(address)	nonpayable
setWeights(uint256,uint256)	nonpayable
snipeMaxBps()	view
snipeSurtaxBps(bytes32)	view
snipeWindowBlocks()	view
surtaxPolicy()	view
sweepLegacyReserve(address,address)	nonpayable
taxExempt(address)	view
totalLifetimeVolume()	view
trackedPools(bytes32)	view
transferOwnership(address)	nonpayable
treasury()	view
vault()	view
volumePerNFT()	view

CauldronToken (15 external/public entries)

Signature	Mutability
TOTAL_SUPPLY()	view
allowance(address,address)	view
approve(address,uint256)	nonpayable
balanceOf(address)	view
birthBlock()	view
burn(address,uint256)	nonpayable
constructor(string,string,uint256,address,uint256)	nonpayable
decimals()	view
generation()	view
name()	view
registry()	view
symbol()	view
totalSupply()	view
transfer(address,uint256)	nonpayable
transferFrom(address,address,uint256)	nonpayable

RedemptionExt (63 external/public entries)

Signature	Mutability
AUTO_MIGRATE_FEE()	view
GEN1_ACTIVE_TOKENS()	view
MAX_NFT_SUPPLY()	view
POOL_FEE()	view
TICK_SPACING()	view
TOTAL_SUPPLY()	view
airdropReserve()	view
airdropWallet()	view
autoMigrate(address)	view

Signature	Mutability
buyTreasury0gFren(uint256)	nonpayable
claimGate()	view
claimed(uint256,address)	view
collectionLedger()	view
currentGeneration()	view
currentToken()	view
donateToReserve(uint256)	nonpayable
emergencyReadyAt()	view
enchantFeeMultBps()	view
factory()	view
floorPerFren()	view
generationCollection(uint256)	view
generationParent(uint256)	view
generationPoolId(uint256)	view
generationPoolKey(uint256)	view
generationPositionId(uint256)	view
generationProposer(uint256)	view
generationReservePositionId(uint256)	view
generationToken(uint256)	view
generationVault(uint256)	view
genesisBaseURI()	view
genesisBonusBps()	view
genesisMode()	view
genesisRenderer()	view
genesisReserveOutstanding()	view
genesisSharePerFren()	view
genesisShares()	view
governor()	view
guardian()	view
hook()	view
igniter()	view
lastSummonAt()	view
materializeLegacyReserve()	nonpayable
mifrens()	view
minLifetime()	view
nextSeedWindow()	view
nftMaxSupply()	view
owner()	view
poolManager()	view
positionManager()	view
primeBuyEth()	view
primeFunder()	view
redeem0gFren(uint256)	nonpayable
redemptionExt()	view
redemptionPaused()	view
renounceOwnership()	nonpayable
reserveTickLower(uint256)	view
reserveTickUpper(uint256)	view
royaltyBps()	view
royaltyDividend()	view

Signature	Mutability
<code>seeder()</code>	view
<code>successor()</code>	view
<code>summoned()</code>	view
<code>transferOwnership(address)</code>	nonpayable

PerpEngine (106 external/public entries)

Signature	Mutability
<code>MAX_OPEN_POSITIONS()</code>	view
<code>POOL_FEE()</code>	view
<code>TICK_SPACING()</code>	view
<code>activeEthDepth()</code>	view
<code>badgesOwed(address)</code>	view
<code>claimLiquidatorBadges(uint256)</code>	nonpayable
<code>claimPayout()</code>	nonpayable
<code>close(uint256,uint256)</code>	nonpayable
<code>constructor(address,address,address,address,address,address,address)</code>	nonpayable
<code>creditPerpFee()</code>	payable
<code>creditPerpFeeToken()</code>	payable
<code>divShareBps()</code>	view
<code>dividend()</code>	view
<code>forceCloseAllDead()</code>	nonpayable
<code>forceCloseDead(uint256)</code>	nonpayable
<code>freeEth()</code>	view
<code>freeToken()</code>	view
<code>fundFromVault()</code>	payable
<code>fundInsurance()</code>	payable
<code>fundPlv()</code>	payable
<code>fundPlvToken(uint256)</code>	nonpayable
<code>fundTokenFromVault(uint256)</code>	nonpayable
<code>fundingDelta(uint256)</code>	view
<code>fundingIndex()</code>	view
<code>fundingRateBpsPerDay()</code>	view
<code>hookAddr()</code>	view
<code>insuranceBps()</code>	view
<code>insuranceEth()</code>	view
<code>insuranceFloor()</code>	view
<code>isLiquidatable(uint256)</code>	view
<code>keeperBps()</code>	view
<code>lastFundingAt()</code>	view
<code>lastObsTs()</code>	view
<code>lastRingTs()</code>	view
<code>lastTick()</code>	view
<code>liqPenaltyBps()</code>	view
<code>liquidate(uint256)</code>	nonpayable
<code>long0iEth()</code>	view
<code>maintenanceBps()</code>	view
<code>markSqrtPriceX96()</code>	view

Signature	Mutability
maxFundingBps()	view
maxLeverage()	view
maxLeverageCeiling()	view
maxLiqBps()	view
maxNotionalBps()	view
maxOiBps()	view
maxUtilBps()	view
mifrens()	view
minCollateral()	view
nextId()	view
nftBeneficiary()	view
obsIndex()	view
observations(uint256)	view
ogDiscountBps()	view
openCount()	view
openFeeBps()	view
openLong(uint8,uint256,uint256)	payable
openShort(uint8,uint256,uint256)	payable
owner()	view
payoutOwed(address)	view
plv()	view
plvToken()	view
poke()	nonpayable
poolManager()	view
positionHealth(uint256)	view
positions(uint256)	view
receive()	payable
registry()	view
renounceOwnership()	nonpayable
selfSweep(address)	nonpayable
setFees(uint256,uint256,uint256,uint256,uint256)	nonpayable
setGuards(uint32,uint256,uint256)	nonpayable
setMinCollateral(uint256)	nonpayable
setNftBeneficiary(address)	nonpayable
setRisk(uint256,uint256,uint256,uint256,uint256,uint256)	nonpayable
setRouting(address,address)	nonpayable
setTiers(uint256[],uint8[])	nonpayable
setTwapWindow(uint32)	nonpayable
setVault(address)	nonpayable
setVaultLimits(uint256,uint256)	nonpayable
setVaultSplit(uint256,uint256)	nonpayable
shortOiToken()	view
skimInsurance(uint256,address)	nonpayable
sweepCursor()	view
sweepLiquidations(address)	nonpayable
syncGeneration()	nonpayable
syncedGeneration()	view
syncedToken()	view
tickCumulative()	view
tierDepthWei(uint256)	view

Signature	Mutability
<code>tierLeverage(uint256)</code>	view
<code>tokYieldCumulative()</code>	view
<code>tokYieldEth()</code>	view
<code>totalEth()</code>	view
<code>totalTokenAssets()</code>	view
<code>transferOwnership(address)</code>	nonpayable
<code>treasury()</code>	view
<code>twapTick()</code>	view
<code>twapWindow()</code>	view
<code>unlockCallback(bytes)</code>	nonpayable
<code>vault()</code>	view
<code>vaultYieldBps()</code>	view
<code>warmup()</code>	view
<code>withdrawPlvTo(uint256,address)</code>	nonpayable
<code>withdrawPlvTokenTo(uint256,address)</code>	nonpayable
<code>withdrawTokYieldTo(uint256,address)</code>	nonpayable

PerpVault (27 external/public entries)

Signature	Mutability
<code>accEthPerTokShare()</code>	view
<code>assetsEth()</code>	view
<code>assetsTok()</code>	view
<code>claimPendingEth()</code>	nonpayable
<code>claimPendingToken()</code>	nonpayable
<code>claimTokYield()</code>	nonpayable
<code>constructor(address,address)</code>	nonpayable
<code>depositEth()</code>	payable
<code>depositToken(uint256)</code>	nonpayable
<code>engine()</code>	view
<code>ethPosition(address)</code>	view
<code>ethShareOf(address)</code>	view
<code>ethShares()</code>	view
<code>lastTokYieldCum()</code>	view
<code>pendingEth()</code>	view
<code>pendingEthOf(address)</code>	view
<code>pendingTok()</code>	view
<code>pendingTokOf(address)</code>	view
<code>pendingTokYield(address)</code>	view
<code>registry()</code>	view
<code>tokRewardDebt(address)</code>	view
<code>tokRewardOwed(address)</code>	view
<code>tokShareOf(address)</code>	view
<code>tokShares()</code>	view
<code>tokenPosition(address)</code>	view
<code>withdrawEth(uint256)</code>	nonpayable
<code>withdrawToken(uint256)</code>	nonpayable

CauldronSeeder (27 external/public entries)

Signature	Mutability
<code>baseWad()</code>	view
<code>complete()</code>	view
<code>constructor(address,address,address)</code>	nonpayable
<code>deployedWad()</code>	view
<code>ethTotal()</code>	view
<code>gen()</code>	view
<code>isComplete()</code>	view
<code>minStepWad()</code>	view
<code>placedWad()</code>	view
<code>poke()</code>	nonpayable
<code>pokeInSwap()</code>	nonpayable
<code>poolManager()</code>	view
<code>positionManager()</code>	view
<code>rangeCount()</code>	view
<code>ranges(uint256)</code>	view
<code>receive()</code>	payable
<code>registry()</code>	view
<code>rescue(address)</code>	nonpayable
<code>seedFloorWad()</code>	view
<code>seeding()</code>	view
<code>startSeed(tuple)</code>	payable
<code>startTs()</code>	view
<code>token()</code>	view
<code>tokenTotal()</code>	view
<code>unlockCallback(bytes)</code>	nonpayable
<code>window()</code>	view
<code>withdrawAll(address)</code>	nonpayable

CauldronGovernor (18 external/public entries)

Signature	Mutability
<code>MAX_NFT_SUPPLY()</code>	view
<code>VOTING_PERIOD()</code>	view
<code>constructor(address)</code>	nonpayable
<code>displayName(uint256)</code>	view
<code>getProposal(uint256)</code>	view
<code>hasProposals()</code>	view
<code>hasVoted(uint256,address)</code>	view
<code>markConsumed(uint256)</code>	nonpayable
<code>mifrens()</code>	view
<code>owner()</code>	view
<code>proposalCount()</code>	view
<code>propose(string,string,uint8,string,address,string,string,uint256,uint256)</code>	nonpayable
<code>registry()</code>	view
<code>renounceOwnership()</code>	nonpayable
<code>setRegistry(address)</code>	nonpayable

Signature	Mutability
<code>transferOwnership(address)</code>	nonpayable
<code>vote(uint256)</code>	nonpayable
<code>winner()</code>	view

CauldronCollection (49 external/public entries)

Signature	Mutability
<code>LIQUIDATOR_ID_BASE()</code>	view
<code>approve(address,uint256)</code>	nonpayable
<code>balanceOf(address)</code>	view
<code>burnFromVault(uint256)</code>	nonpayable
<code>configurator()</code>	view
<code>constructor(string,string,address,address,uint256,uint8,string,address,uint256)</code>	nonpayable
<code>custodyTransfer(address,address,uint256)</code>	nonpayable
<code>deployer()</code>	view
<code>getApproved(uint256)</code>	view
<code>getTransferValidationFunction()</code>	pure
<code>getTransferValidator()</code>	view
<code>isApprovedForAll(address,address)</code>	view
<code>isLiquidator(uint256)</code>	view
<code>liquidatorTrait(uint256)</code>	view
<code>liquidatorMinted()</code>	view
<code>liquidatorMinter()</code>	view
<code>liquidatorURI()</code>	view
<code>maxSupply()</code>	view
<code>mint(address)</code>	nonpayable
<code>mintBlockOf(uint256)</code>	view
<code>mintLiquidator(address)</code>	nonpayable
<code>minter()</code>	view
<code>mode()</code>	view
<code>name()</code>	view
<code>ownerOf(uint256)</code>	view
<code>rarityCumBps(uint256)</code>	view
<code>rarityOf(uint256)</code>	view
<code>renderer()</code>	view
<code>reveal(uint256)</code>	nonpayable
<code>revealed(uint256)</code>	view
<code>royaltyInfo(uint256,uint256)</code>	view
<code>safeTransferFrom(address,address,uint256)</code>	nonpayable
<code>safeTransferFrom(address,address,uint256,bytes)</code>	nonpayable
<code>setApprovalForAll(address,bool)</code>	nonpayable
<code>setLiquidatorMinter(address)</code>	nonpayable
<code>setLiquidatorURI(string)</code>	nonpayable
<code>setMetadata(uint8,address,string)</code>	nonpayable
<code>setRarityOdds(uint16[4])</code>	nonpayable
<code>setRoyalty(address,uint96)</code>	nonpayable
<code>setTransferValidator(address)</code>	nonpayable
<code>setVault(address)</code>	nonpayable

Signature	Mutability
<code>supportsInterface(bytes4)</code>	view
<code>symbol()</code>	view
<code>tokenURI(uint256)</code>	view
<code>totalMinted()</code>	view
<code>transferFrom(address,address,uint256)</code>	nonpayable
<code>transferValidator()</code>	view
<code>unrevealedURI()</code>	view
<code>vault()</code>	view

MiFrensGenesis (82 external/public entries)

Signature	Mutability
<code>CLOCK_MODE()</code>	view
<code>GENESIS_SUPPLY()</code>	view
<code>LIQUIDATOR_ID_BASE()</code>	view
<code>MAX_PER_WALLET()</code>	view
<code>MAX_SUPPLY()</code>	view
<code>PRICE()</code>	view
<code>approve(address,uint256)</code>	nonpayable
<code>balanceOf(address)</code>	view
<code>burnFromVault(uint256)</code>	nonpayable
<code>cancelPresale()</code>	nonpayable
<code>cancelled()</code>	view
<code>clock()</code>	view
<code>constructor(string,string,uint256,uint256,uint256,uint256,string)</code>	nonpayable
<code>custodyTransfer(address,address,uint256)</code>	nonpayable
<code>delegate(address)</code>	nonpayable
<code>delegateBySig(address,uint256,uint256,uint8,bytes32,bytes32)</code>	nonpayable
<code>delegates(address)</code>	view
<code>deployer()</code>	view
<code>dividend()</code>	view
<code>eip712Domain()</code>	view
<code>everMoved(uint256)</code>	view
<code>finalize()</code>	nonpayable
<code>finalized()</code>	view
<code>finalizer()</code>	view
<code>getApproved(uint256)</code>	view
<code>getPastTotalSupply(uint256)</code>	view
<code>getPastVotes(address,uint256)</code>	view
<code>getTransferValidationFunction()</code>	pure
<code>getTransferValidator()</code>	view
<code>getVotes(address)</code>	view
<code>isApprovedForAll(address,address)</code>	view
<code>isGenesis(uint256)</code>	view
<code>isLiquidatoor(uint256)</code>	view
<code>liquidatoorTrait(uint256)</code>	view
<code>liquidatorMinted()</code>	view
<code>liquidatorMinter()</code>	view

Signature	Mutability
liquidatorURI()	view
maxSupply()	view
mint(address)	nonpayable
mint(uint256)	payable
mintBlockOf(uint256)	view
mintLiquidator(address)	nonpayable
minted()	view
minter()	view
mode()	view
name()	view
nonces(address)	view
ogTrait(uint256)	view
ownerOf(uint256)	view
paid(address)	view
rarityCumBps(uint256)	view
rarityOf(uint256)	view
refund()	nonpayable
registry()	view
remaining()	view
renderer()	view
reveal(uint256)	nonpayable
revealed(uint256)	view
royaltyInfo(uint256,uint256)	view
safeTransferFrom(address,address,uint256)	nonpayable
safeTransferFrom(address,address,uint256,bytes)	nonpayable
setApprovalForAll(address,bool)	nonpayable
setDividend(address)	nonpayable
setFinalizer(address)	nonpayable
setLiquidatorMinter(address)	nonpayable
setLiquidatorURI(string)	nonpayable
setMetadata(uint8,address,string)	nonpayable
setMinter(address)	nonpayable
setRarityOdds(uint16[4])	nonpayable
setRegistry(address)	nonpayable
setRoyalty(address,uint96)	nonpayable
setTransferValidator(address)	nonpayable
setVault(address)	nonpayable
soldOut()	view
supportsInterface(bytes4)	view
symbol()	view
tokenURI(uint256)	view
totalMinted()	view
transferFrom(address,address,uint256)	nonpayable
transferValidator()	view
unrevealedURI()	view
vault()	view

MiFrensDividend (24 external/public entries)

Signature	Mutability
MAX_TOKEN()	view
SHARES()	view
accPerShare()	view
activeShares()	view
castMany(uint256[])	nonpayable
castSpell(uint256)	nonpayable
claim(uint256)	nonpayable
claimMany(uint256[])	nonpayable
constructor(address,address)	nonpayable
debtOf(uint256)	view
enchantedBy(uint256)	view
isEnchanted(uint256)	view
mifrens()	view
onMiFrenTransfer(uint256,address)	nonpayable
owed(address)	view
pending(uint256)	view
receive()	payable
registry()	view
residual()	view
setRegistry(address)	nonpayable
totalClaimed()	view
totalDeposited()	view
treasury()	view
withdrawOwed()	nonpayable

CollectionLedger (13 external/public entries)

Signature	Mutability
buyback(uint256,uint256,uint256)	nonpayable
constructor(address)	nonpayable
credit(uint256,uint256)	nonpayable
crystallize(uint256,uint256,uint256)	nonpayable
crystallized(uint256)	view
entitledTokens(uint256)	view
floorPerNFT(uint256,uint256)	view
frozenSupply(uint256)	view
outstanding(uint256,uint256)	view
redeem(uint256,uint256)	nonpayable
registry()	view
retired(uint256)	view
totalEntitled()	view

MigrationVesting (19 external/public entries)

Signature	Mutability
MAX_WINDOW()	view
MIN_WINDOW()	view

Signature	Mutability
<code>claim()</code>	nonpayable
<code>claimFor(address)</code>	nonpayable
<code>claimable(address)</code>	view
<code>constructor(address,address,uint64,address)</code>	nonpayable
<code>grantAt(address,uint256)</code>	view
<code>grantCount(address)</code>	view
<code>locked(address)</code>	view
<code>owner()</code>	view
<code>registry()</code>	view
<code>renounceOwnership()</code>	nonpayable
<code>setStakerOracle(address)</code>	nonpayable
<code>setVestWindow(uint64)</code>	nonpayable
<code>stakerOracle()</code>	view
<code>startVest(uint256,uint256)</code>	nonpayable
<code>transferOwnership(address)</code>	nonpayable
<code>vestBatch(uint256,address[])</code>	nonpayable
<code>vestWindow()</code>	view

CauldronVault (11 external/public entries)

Signature	Mutability
<code>close()</code>	nonpayable
<code>closed()</code>	view
<code>collection()</code>	view
<code>constructor(address,address,uint256)</code>	nonpayable
<code>floorOffset()</code>	view
<code>floorPerNFT()</code>	view
<code>outstanding()</code>	view
<code>receive()</code>	payable
<code>redeem(uint256)</code>	nonpayable
<code>redeemed()</code>	view
<code>registry()</code>	view

CauldronFactory (2 external/public entries)

Signature	Mutability
<code>deployBrew(tuple)</code>	nonpayable
<code>deployVault(address,address,uint256)</code>	nonpayable

CauldronGachaRouter (19 external/public entries)

Signature	Mutability
<code>MAX_LOOPS()</code>	view
<code>MAX_MINTS_PER_CALL()</code>	view
<code>POOL_FEE()</code>	view

Signature	Mutability
TICK_SPACING()	view
constructor(address,address,address,address)	nonpayable
hook()	view
hookAddr()	view
openReady(uint256)	nonpayable
owner()	view
play(uint256,uint256,uint256,uint256)	payable
playChurn(uint256,uint256)	payable
playLiq(uint256,uint256,uint256,uint256,uint256[])	payable
poolManager()	view
receive()	payable
registry()	view
renounceOwnership()	nonpayable
rescueETH(address,uint256)	nonpayable
transferOwnership(address)	nonpayable
unlockCallback(bytes)	nonpayable

LaunchSniper (7 external/public entries)

Signature	Mutability
constructor(address)	nonpayable
launch(address,address,address,address,uint256,uint256)	payable
owner()	view
receive()	payable
renounceOwnership()	nonpayable
sweep(address)	nonpayable
transferOwnership(address)	nonpayable

PerpStakerOracle (3 external/public entries)

Signature	Mutability
constructor(address)	nonpayable
isInstant(address)	view
perpVault()	view

RoyaltyRouter (3 external/public entries)

Signature	Mutability
constructor(address)	nonpayable
hook()	view
receive()	payable

DefaultFeeRouter (1 external/public entries)

Signature	Mutability
<code>route(uint256,address,address,uint256,uint256)</code>	pure

PoolOps (1 external/public entries)

Signature	Mutability
<code>creatureFor(uint256)</code>	pure

ReserveLib (0 external/public entries)

Signature	Mutability

SeedLib (0 external/public entries)

Signature	Mutability